

Lecture 7

Mixture of Gaussians Linear Regression

EECS 189/289, Spring 2026 @ UC Berkeley

Assigned Book Chapters:

Linear Regression:

Chapter 1.[2.1-2.3]

Chapter 4.[1.1, 1.4]

Roadmap

- Mixture of Gaussians
- Linear Regression
 - Probabilistic/MLE view
 - Geometric view

Roadmap

- Mixture of Gaussians
- Linear Regression
 - Probabilistic/MLE view
 - Geometric view

Recap from last class

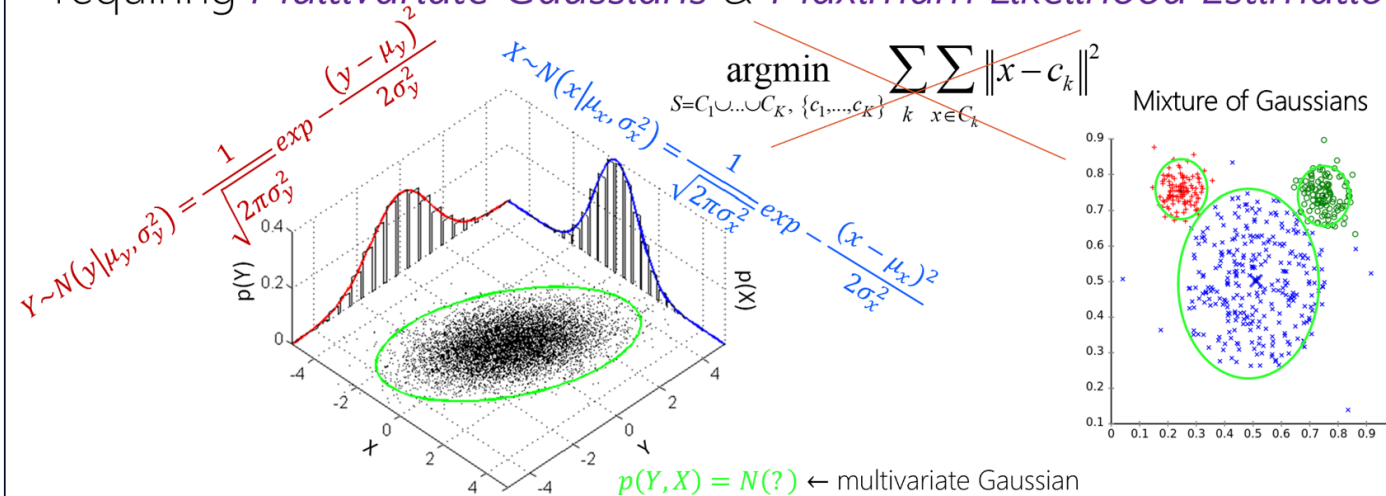
In Discussion, you will work out how to do MLE for a MVN distribution.

The multivariate extension is: $x \in \mathbb{R}^d, u \in \mathbb{R}^d$ and $\Sigma \in \mathbb{R}^{d \times d}$ and PSD.

$$X \sim \mathcal{N}(x | \mu, \Sigma) = \frac{1}{(2\pi)^{D/2}} \frac{1}{|\Sigma|^{1/2}} \exp\left(-\frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu)\right)$$

$\sqrt{|2\pi\Sigma|}$

We will need to estimate parameters of a *Mixture of Gaussians*, requiring *Multivariate Gaussians* & *Maximum Likelihood Estimation*.



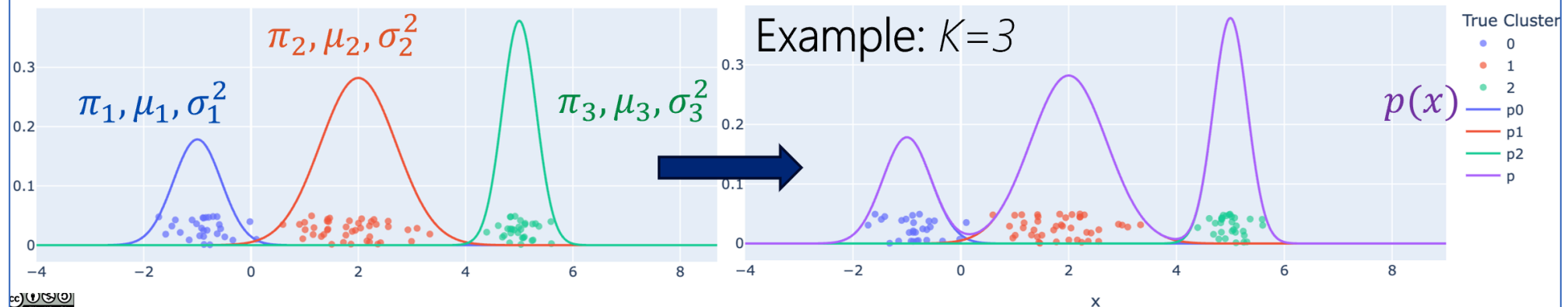
Recap from last class

Mixture of Gaussians

The Gaussian mixture model, $p(\mathbf{x})$, defines the probability of sampling a D -dimensional vector $\mathbf{x}$ as a **weighted combination** of Gaussians:

$$p(\mathbf{x} \mid \alpha, \mu_1, \Sigma_1, \dots, \mu_K, \Sigma_K) = \sum_{k=1}^K \alpha_k \mathcal{N}(\mathbf{x} \mid \mu_k, \Sigma_k)$$

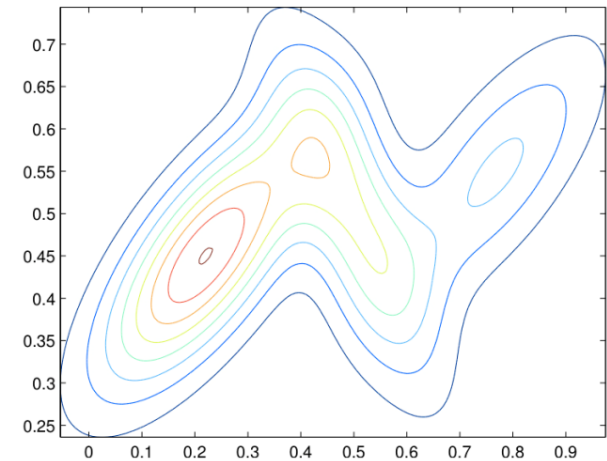
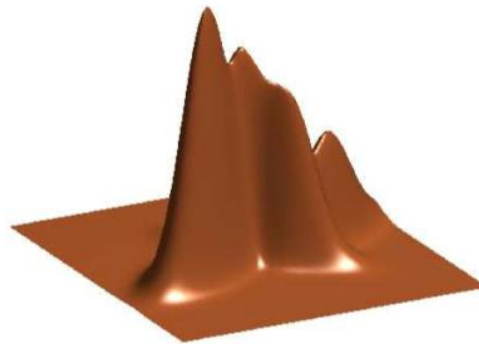
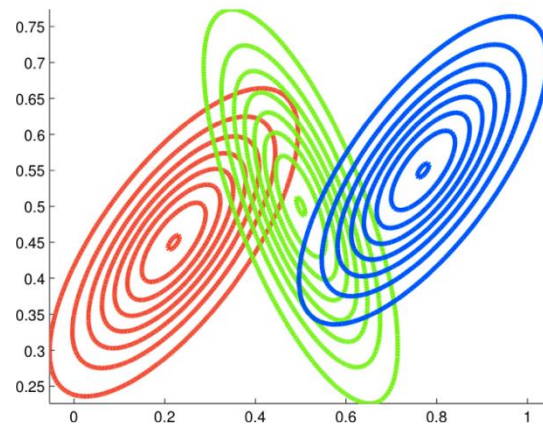
where $\sum_{k=1}^K \alpha_k = 1$ is a probability distribution over clusters.



Mixture of Gaussians (MoG) for clustering

- Each cluster is now represented by a Gaussian $N(\mathbf{x}_i | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$, with two free parameters.
- Now we can write down a **clustering model likelihood** and perform MLE!

$$p(\mathbf{x} | \alpha, \mu_1, \Sigma_1, \dots, \mu_K, \Sigma_K) = \sum_{k=1}^K \alpha_k \mathcal{N}(\mathbf{x} | \mu_k, \Sigma_k)$$

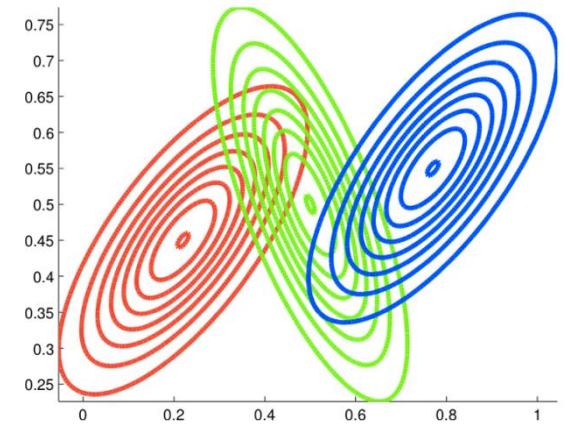


Mixture of Gaussians (MoG) likelihood for one $\mathbf{x}_i$

- Let $z_i \in \{1, \dots, K\}$ be the hidden/unobserved assigned cluster to $\mathbf{x}_i$. We don't know its value, so have to marginalize it (sum it out):

$$L_i = p(\mathbf{x}_i | \boldsymbol{\theta}) = \sum_{k=1}^K p(\mathbf{x}_i, z_i = k | \boldsymbol{\theta})$$

"latent variable"



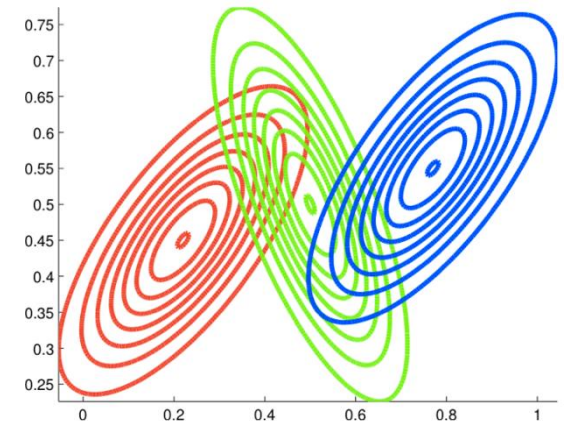
$$p(\mathbf{x} | \boldsymbol{\alpha}, \mu_1, \Sigma_1, \dots, \mu_K, \Sigma_K) = \sum_{k=1}^K \alpha_k \mathcal{N}(\mathbf{x} | \mu_k, \Sigma_k)$$

Mixture of Gaussians (MoG) likelihood for one $\mathbf{x}_i$

- Let $z_i \in \{1, \dots, K\}$ be the hidden/unobserved assigned cluster to $\mathbf{x}_i$. We don't know its value, so have to marginalize it (sum it out):

$$\begin{aligned} L_i &= p(\mathbf{x}_i | \boldsymbol{\theta}) = \sum_{k=1}^K p(\mathbf{x}_i, z_i = k | \boldsymbol{\theta}) \\ &= \sum_{k=1}^K p(\mathbf{x}_i | z_i = k, \boldsymbol{\theta}) p(z_i = k | \boldsymbol{\theta}) \end{aligned}$$

"latent variable"

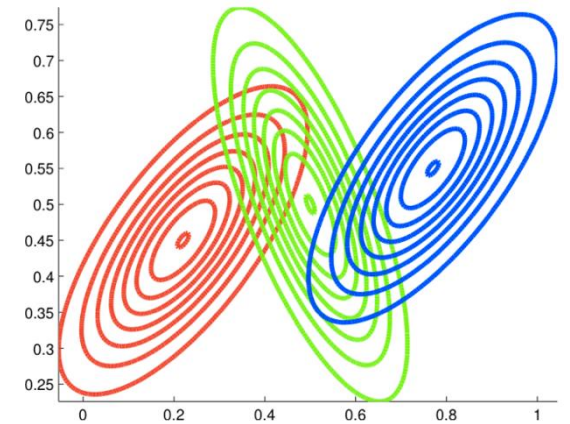


$$p(\mathbf{x} | \boldsymbol{\alpha}, \mu_1, \Sigma_1, \dots, \mu_K, \Sigma_K) = \sum_{k=1}^K \alpha_k \mathcal{N}(\mathbf{x} | \mu_k, \Sigma_k)$$

Mixture of Gaussians (MoG) likelihood for one $\mathbf{x}_i$

- Let $\mathbf{z}_i \in \{1, \dots, K\}$ be the hidden/unobserved assigned cluster to $\mathbf{x}_i$. We don't know its value, so have to marginalize it (sum it out):

$$\begin{aligned} L_i &= p(\mathbf{x}_i | \boldsymbol{\theta}) = \sum_{k=1}^K p(\mathbf{x}_i, \mathbf{z}_i = k | \boldsymbol{\theta}) \\ &= \sum_{k=1}^K p(\mathbf{x}_i | \mathbf{z}_i = k, \boldsymbol{\theta}) p(\mathbf{z}_i = k | \boldsymbol{\theta}) \\ &= \sum_{k=1}^K N(\mathbf{x}_i | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) p(\mathbf{z}_i = k | \boldsymbol{\theta}) \end{aligned}$$



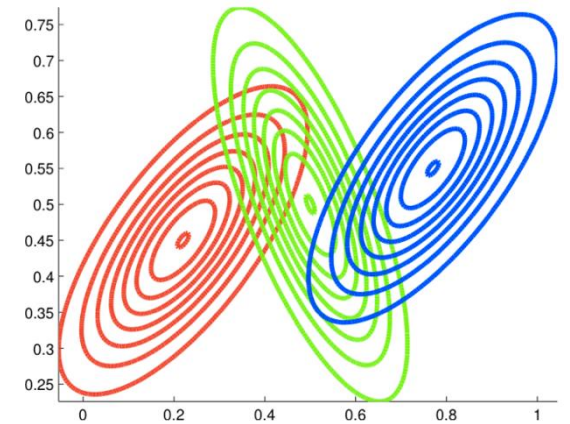
$$p(\mathbf{x} | \boldsymbol{\alpha}, \boldsymbol{\mu}_1, \boldsymbol{\Sigma}_1, \dots, \boldsymbol{\mu}_K, \boldsymbol{\Sigma}_K) = \sum_{k=1}^K \alpha_k \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

Mixture of Gaussians (MoG) likelihood for one $\mathbf{x}_i$

- Let $\mathbf{z}_i \in \{1, \dots, K\}$ be the hidden/unobserved assigned cluster to $\mathbf{x}_i$. We don't know its value, so have to marginalize it (sum it out):

$$\begin{aligned} L_i &= p(\mathbf{x}_i | \boldsymbol{\theta}) = \sum_{k=1}^K p(\mathbf{x}_i, \mathbf{z}_i = k | \boldsymbol{\theta}) \\ &= \sum_{k=1}^K p(\mathbf{x}_i | \mathbf{z}_i = k, \boldsymbol{\theta}) p(\mathbf{z}_i = k | \boldsymbol{\theta}) \\ &= \sum_{k=1}^K N(\mathbf{x}_i | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) p(\mathbf{z}_i = k | \boldsymbol{\theta}) \\ &= \sum_{k=1}^K N(\mathbf{x}_i | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \alpha_k \end{aligned}$$

where $\alpha_k \equiv p(\mathbf{z}_i = k)$ and $\sum_k \alpha_k = 1$



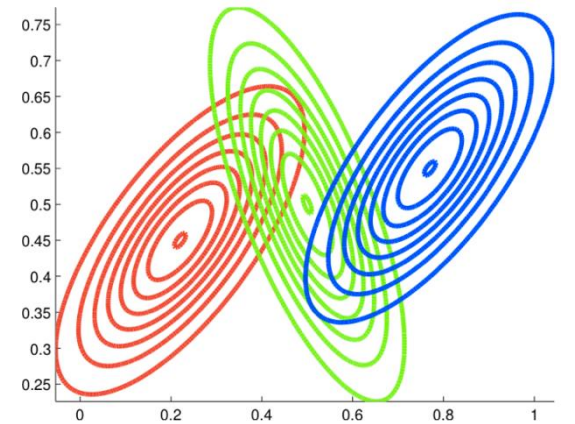
$$p(\mathbf{x} | \alpha, \mu_1, \Sigma_1, \dots, \mu_K, \Sigma_K) = \sum_{k=1}^K \alpha_k \mathcal{N}(\mathbf{x} | \mu_k, \Sigma_k)$$

Mixture of Gaussians (MoG) likelihood for one $\mathbf{x}_i$

- Let $\mathbf{z}_i \in \{1, \dots, K\}$ be the hidden/unobserved assigned cluster to $\mathbf{x}_i$. We don't know its value, so have to marginalize it (sum it out):

$$\begin{aligned} L_i &= p(\mathbf{x}_i | \boldsymbol{\theta}) = \sum_{k=1}^K p(\mathbf{x}_i, \mathbf{z}_i = k | \boldsymbol{\theta}) \\ &= \sum_{k=1}^K p(\mathbf{x}_i | \mathbf{z}_i = k, \boldsymbol{\theta}) p(\mathbf{z}_i = k | \boldsymbol{\theta}) \\ &= \sum_{k=1}^K N(\mathbf{x}_i | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) p(\mathbf{z}_i = k | \boldsymbol{\theta}) \\ &= \sum_{k=1}^K N(\mathbf{x}_i | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \alpha_k \end{aligned}$$

where $\alpha_k \equiv p(\mathbf{z}_i = k)$ and $\sum_k \alpha_k = 1$



$$p(\mathbf{x} | \alpha, \mu_1, \Sigma_1, \dots, \mu_K, \Sigma_K) = \sum_{k=1}^K \alpha_k \mathcal{N}(\mathbf{x} | \mu_k, \Sigma_k)$$

- The parameters we want to learn are $\boldsymbol{\theta} \equiv \{\mu_k, \Sigma_k, \alpha_k\}$.
- α_k are called the "mixing weights".
- Now we can do MLE with $LL = \log \prod_i L_i = \sum_i \log L_i = \sum_i LL_i$.

Mixture of Gaussians (MoG) likelihood for one x_i

- Let $z_i \in \{1, \dots, K\}$ to x_i . We can write the likelihood (sum it out)

$$LL_i = \log \sum_{k=1}^K N(x_i | \mu_k, \Sigma_k) \alpha_k$$

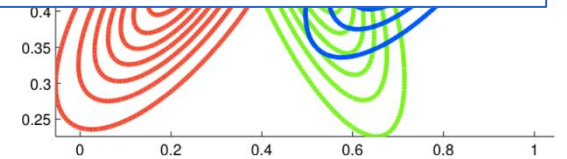
$$L_i = \Rightarrow LL = \sum_i LL_i = \sum_i \log \sum_{k=1}^K N(x_i | \mu_k, \Sigma_k) \alpha_k$$

=

$$= \sum_{k=1}^K N(x_i | \mu_k, \Sigma_k) p(z_i = k | \theta)$$

$$= \sum_{k=1}^K N(x_i | \mu_k, \Sigma_k) \alpha_k$$

where $\alpha_k \equiv p(z_i = k)$ and $\sum_k \alpha_k = 1$



- The parameters we want to learn are $\theta \equiv \{\mu_k, \Sigma_k, \alpha_k\}$.
- α_k are called the "mixing weights".
- Now we can do MLE with $LL = \log \prod_i L_i = \sum_i \log L_i = \sum_i LL_i$.

Mixture of Gaussians (MoG) likelihood for one x_i

$$LL = \sum_i LL_i = \sum_i \log \sum_{k=1}^K N(x_i | \mu_k, \Sigma_k) \alpha_k$$

- But what difficulty will we have?!?
- It will not be closed-form (analytical) solution.
- We will need an iterative optimization algorithm: gradient descent, which will get taught later (brief synopsis today):

$$\frac{dLL}{d\mu_k} = ?$$

$$\frac{dLL}{d\alpha_k} = ?$$

$$\frac{dLL}{d\Sigma_k} = ?$$

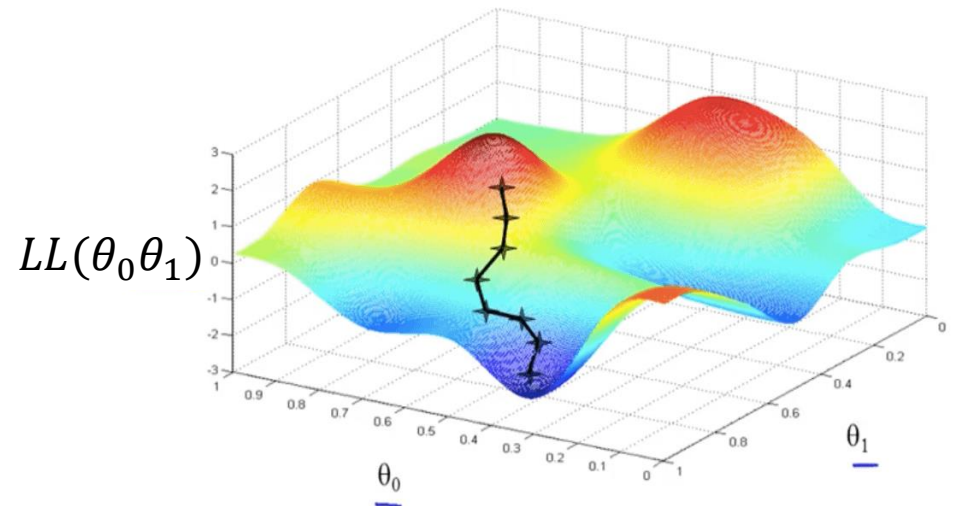
Recall from multivariable calculus that the *gradient* of $f(\mathbf{x})$ is the vector:

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_d} \end{bmatrix}$$

which is the direction that leads to a maximal increase of $f(\mathbf{x})$.

Gradient descent optimization algorithm in a nutshell:

- Initialize loop counter: $t = 0$.
- Initialize params: choose $\theta_{t=0} \in \mathbb{R}^d$
- Choose *step size*, $\alpha > 0$.
- Until convergence of θ_t do:
 - $\theta_{t+1} \leftarrow \theta_t + \alpha \nabla_{\theta} LL(\theta_t)$
 - $t \leftarrow t + 1$

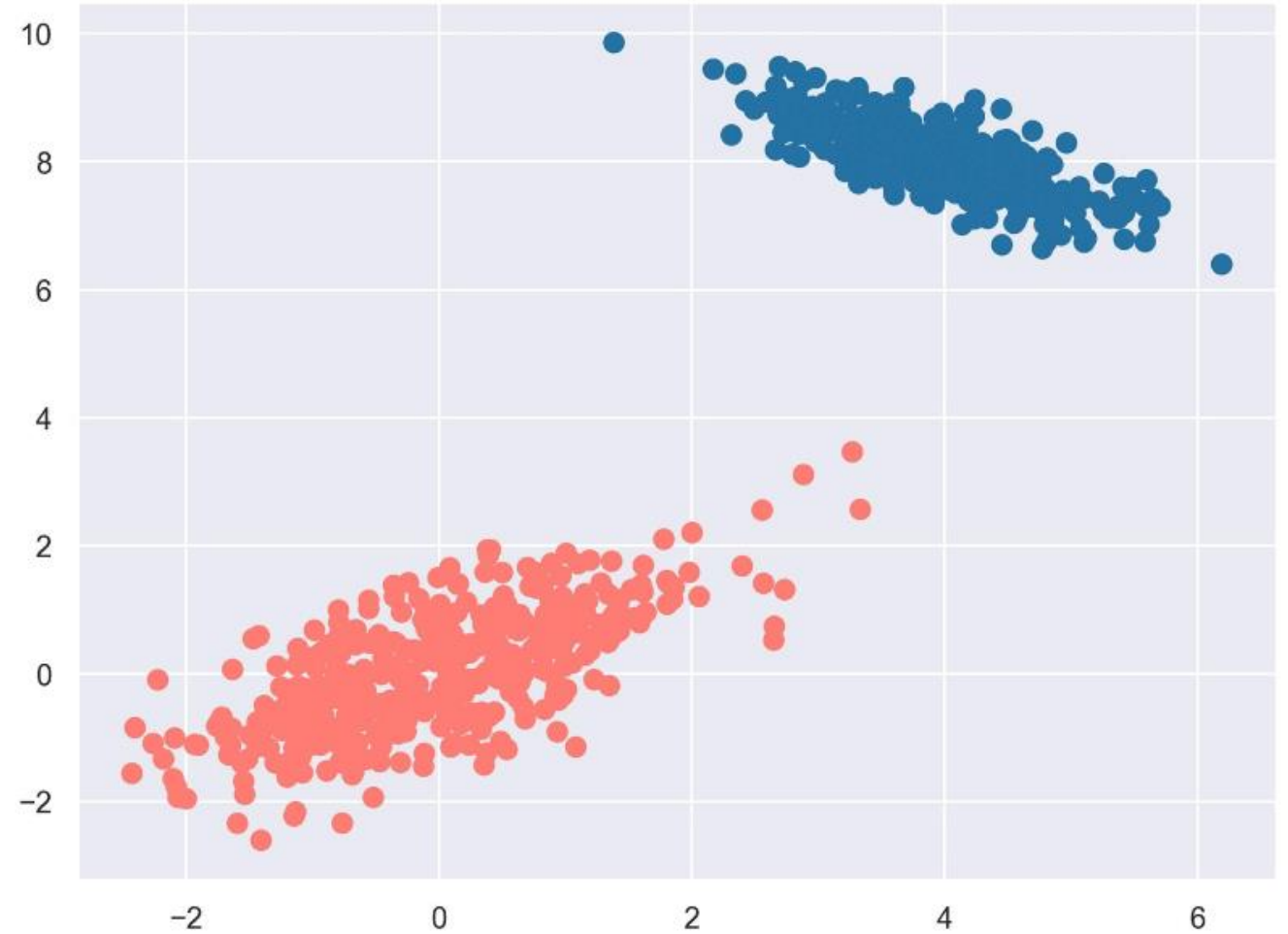


Mixture of Gaussians

$$LL = \sum_i \log N(x_i | \mu_k, \Sigma_k) \alpha_k$$

$$\begin{matrix} x^T & \Sigma^{-1} & x \\ 1 \times d & d \times d & d \times 1 \end{matrix} = \text{constant}$$

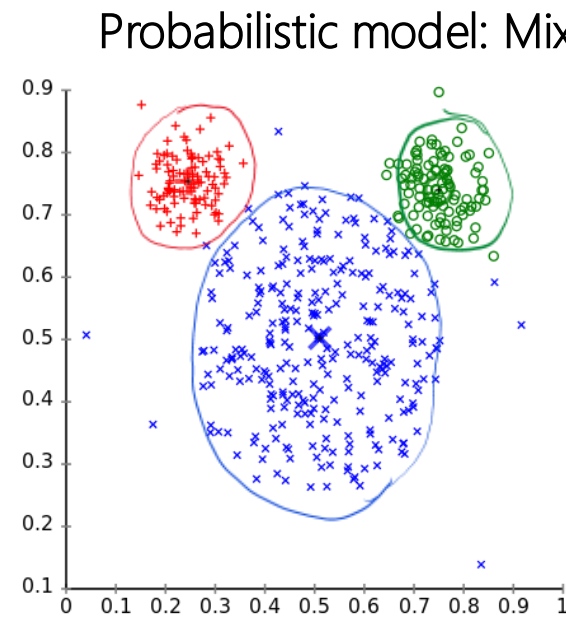
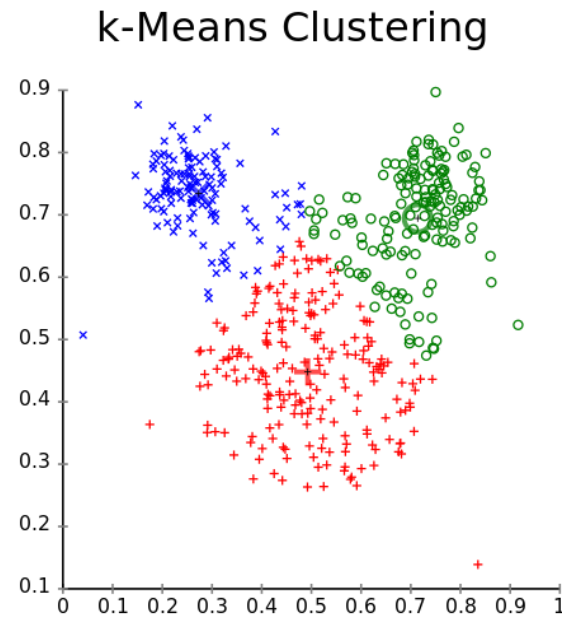
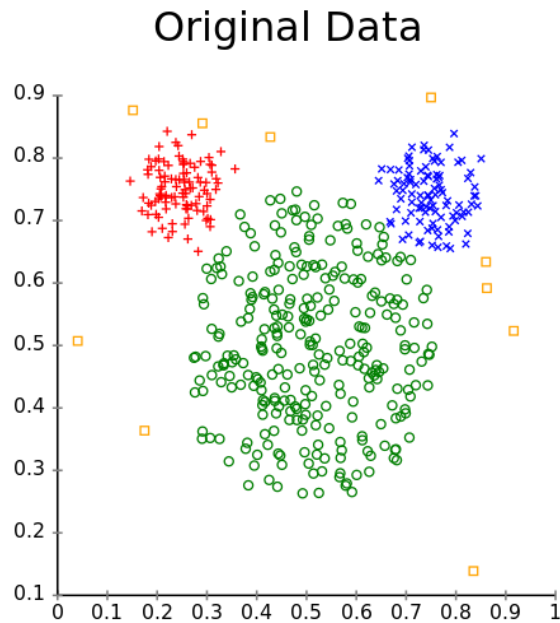
Observed Data



Mixture of Gaussians (MoG) likelihood for one x_i

- We learned how to cast k-means as a probabilistic model, and learned how to do MLE.

$$p(\mathbf{x} \mid \pi, \mu_1, \Sigma_1, \dots, \mu_K, \Sigma_K) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} \mid \mu_k, \Sigma_k) = 0.5 \times \mathcal{N}(\mathbf{x} \mid \mu_k, \Sigma_k) + 0.3 \times \mathcal{N}(\mathbf{x} \mid \mu_k, \Sigma_k) + 0.2 \times \mathcal{N}(\mathbf{x} \mid \mu_k, \Sigma_k)$$



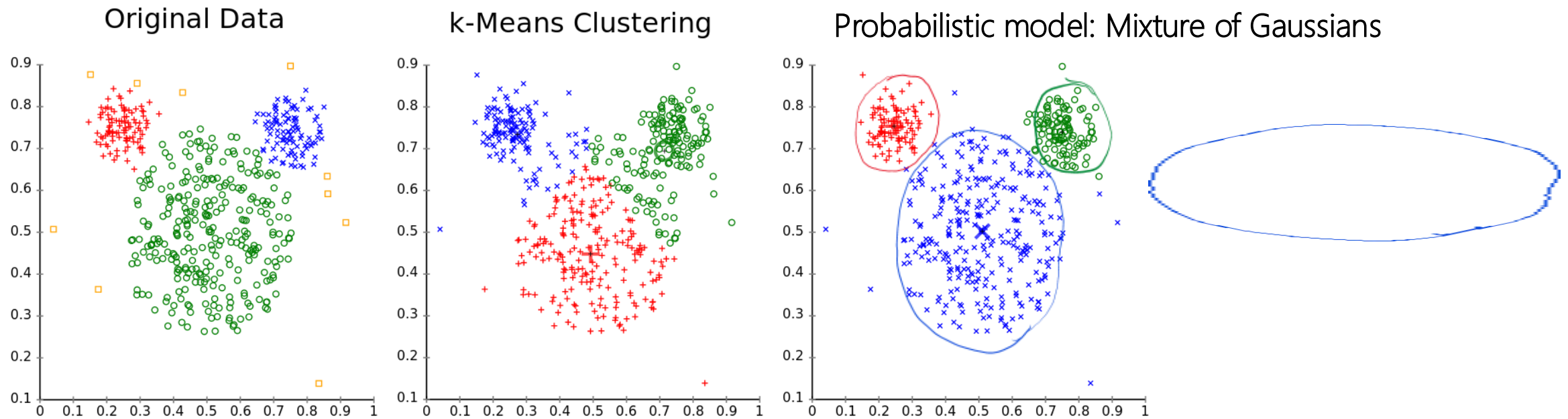
$$\underset{S=C_1 \sqcup \dots \sqcup C_K, \{c_1, \dots, c_K\}}{\operatorname{argmin}} \sum_k \sum_{x \in C_k} \|x - c_k\|^2$$

$$p(x; \mu, \Sigma) = \frac{1}{(2\pi)^{n/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu) \right)$$

Mixture of Gaussians (MoG) likelihood for one x_i

- The zero noise limit of Mixture of Gaussians yields K-means.
- Different covariance per cluster is usually helpful, including here.

$$p(\mathbf{x} \mid \pi, \mu_1, \Sigma_1, \dots, \mu_K, \Sigma_K) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} \mid \mu_k, \Sigma_k) = 0.5 \times \mathcal{N}(\mathbf{x} \mid \mu_k, \Sigma_k) + 0.3 \times \mathcal{N}(\mathbf{x} \mid \mu_k, \Sigma_k) + 0.2 \times \mathcal{N}(\mathbf{x} \mid \mu_k, \Sigma_k)$$



$$\underset{S=C_1 \sqcup \dots \sqcup C_K, \{c_1, \dots, c_K\}}{\operatorname{argmin}} \sum_k \sum_{x \in C_k} \|x - c_k\|^2$$

$$p(x; \mu, \Sigma) = \frac{1}{(2\pi)^{n/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu) \right)$$

K-means vs. Mixture of Gaussians

- MoG: explicit assumptions in the form of statistical distributions.
- Thus easier to generalize, while understanding assumptions (*e.g.*, oblong/elliptical shaped clusters).
- Can derive principled objective in the form of a likelihood, which involves marginalizing over the hidden/latent variable (cluster assignment).
- There is a special form of MLE for these latent variables called Expectation-Maximization (see textbook 15.3-15.3.2 if interested)

Alternative uses of MoG beyond clustering

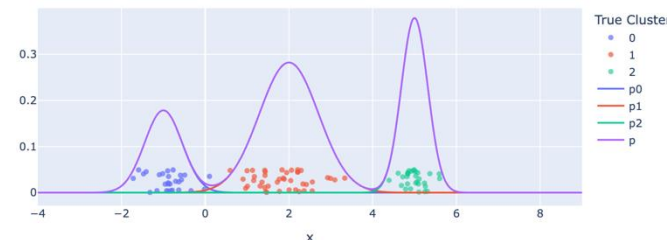
Once we have estimated the values of $\theta = \{\mu_k, \Sigma_k, \alpha_k\}$ from the training data, we can make calls to $p(\mathbf{x}|\theta)$, for any data point in the training data or otherwise.

- So we have also performed *density estimation*: $p_{\hat{\theta}}(\mathbf{x})$.

We can also use it to generate data, $\mathbf{x} \sim p(\mathbf{x}|\theta)$.

- So we have a *generative model*:
 1. For each point, j , sample cluster $c_j \sim \text{multinomial}(\{\alpha_k\})$.
 2. Then sample from the corresponding Gaussian.

$$p(\mathbf{x} | \alpha, \mu_1, \Sigma_1, \dots, \mu_K, \Sigma_K) = \sum_{k=1}^K \alpha_k \mathcal{N}(\mathbf{x} | \mu_k, \Sigma_k)$$

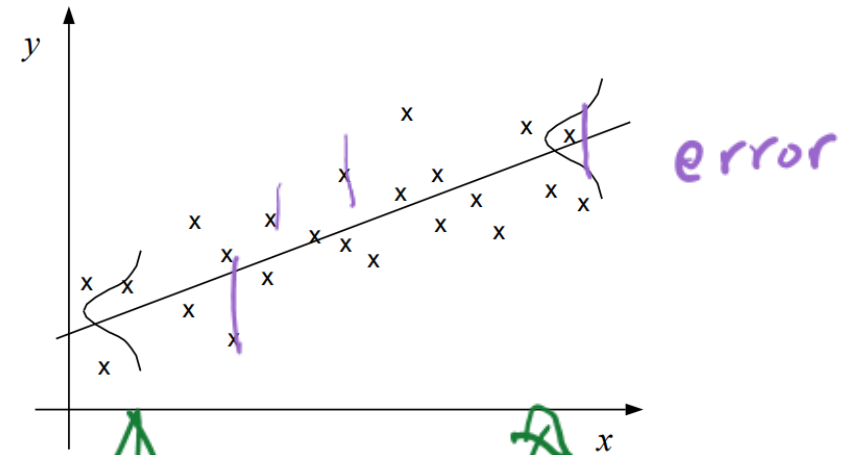


Roadmap

- Mixture of Gaussians
- Linear Regression
 - Probabilistic/MLE view
 - Geometric view

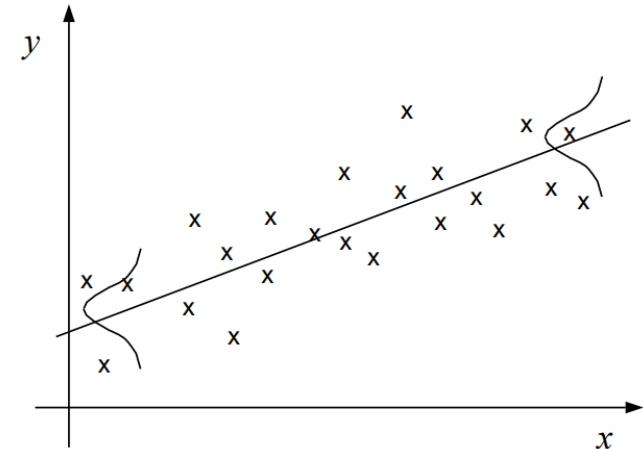
Regression

- *Supervised learning*: data pairs $D = \{(x_i, y_i)\}$, where, x_i may be discrete and/or continuous.
- *Regression*: label, y_i , is a real-valued, e.g., $y_i \in \mathbb{R}$.
- Formally, want $p(y|x)$, the conditional pdf.
- "Point" prediction is then $\hat{y} = E_Y[p(Y|X = x)]$.



Regression examples

- Covid infection rates from zip code and vaccination rate, etc.
- How much a particular protein will bind to a drug target.
- A person's blood pressure from their genetics.
- Tracking - object location in video at the next time-step.
- Housing prices, crime rate|where $\vec{x} = (x_1, x_2, \dots, x_D)^T \in \mathbb{R}^D$.
- Earliest regression: Legendre in 1805, and Gauss in 1809, both estimating orbits of bodies about the sun.



Regression examples

- Covid infection rates from zip code and vaccination rate, etc.
- How much a particular protein will bind to a drug target.
- A person's blood pressure from their genetics.
- Tracking - object location in video at the next time-step.
- Housing prices, crime rate|where $\vec{x} = (x_1, x_2, \dots, x_D)^T \in \mathbb{R}^D$.
- Earliest regression: Legendre in 1805, and Gauss in 1809, both estimating orbits of bodies about the sun.

In higher dimensions, the model
(predicted values) forms a hyperplane in

$$y(\vec{x}|\vec{w}) = w_0 + w_1x_1 + \dots + w_Dx_D$$

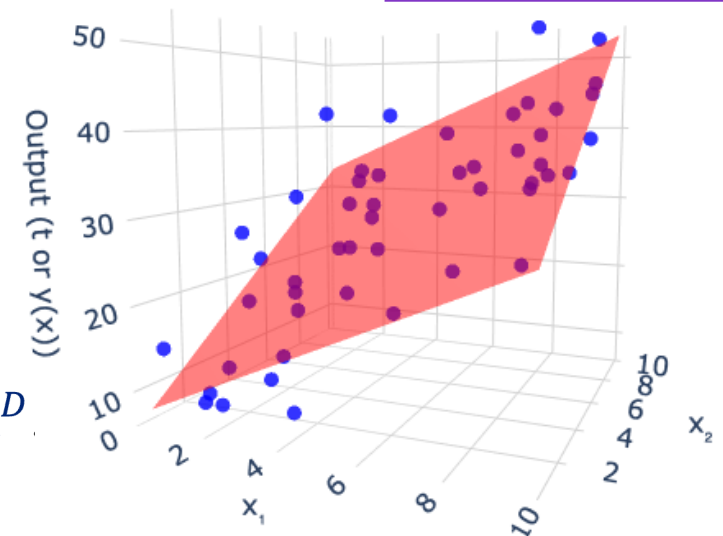
Predicted output

Linear combination of parameters

$w_0, w_1, \dots, w_D$, each scaled by x_d 's.

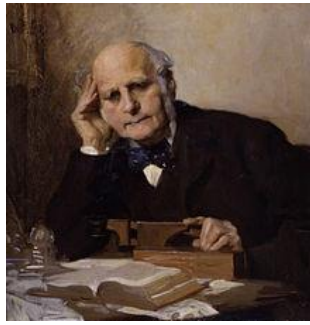
|where $\vec{x} = (x_1, x_2, \dots, x_D)^T \in \mathbb{R}^D$.

[Interactive link](#)

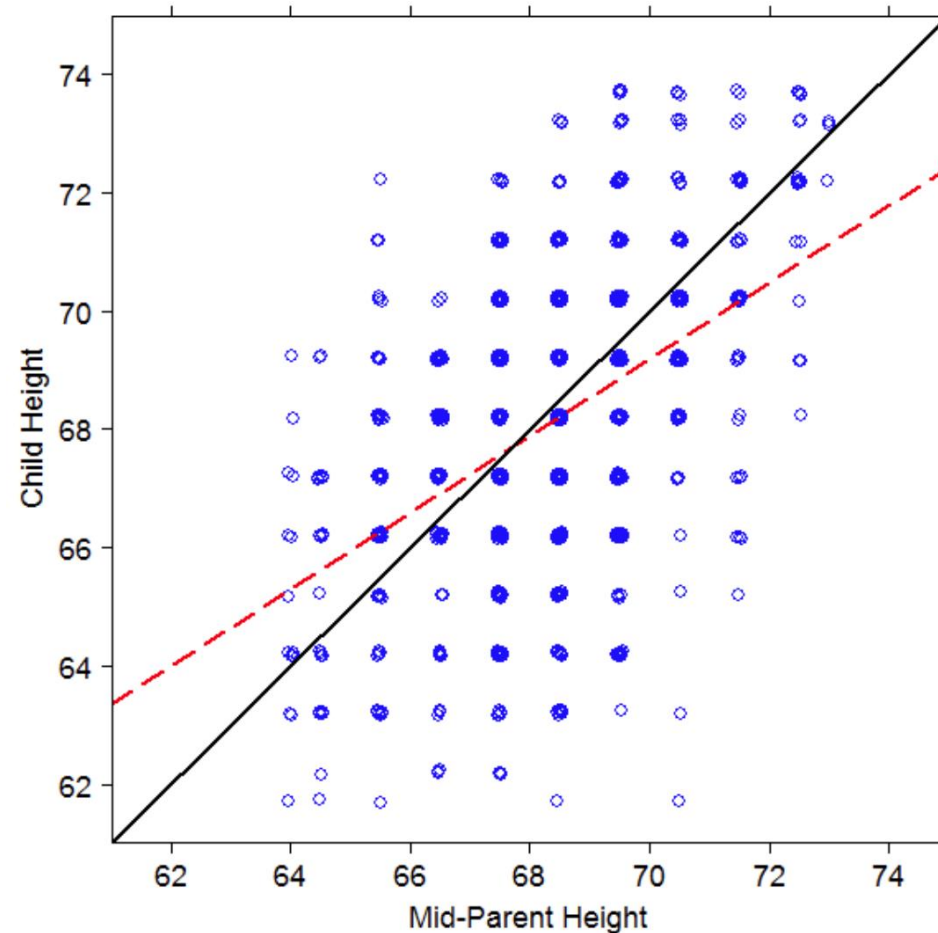


History of the term "Regression"

Sir Francis Galton (1822-1911) *"regression to the mean"*.

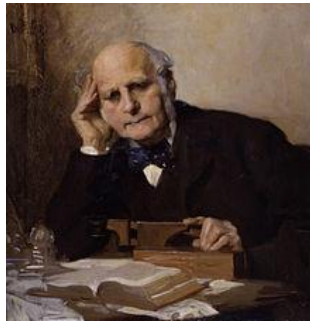


*"It appeared from these experiments that the offspring did not tend to resemble their parents in size, but **always to be more mediocre than they** – to be smaller than the parents, if the parents were large; to be larger than the parents, if the parents were small."*



History of the term "Regression"

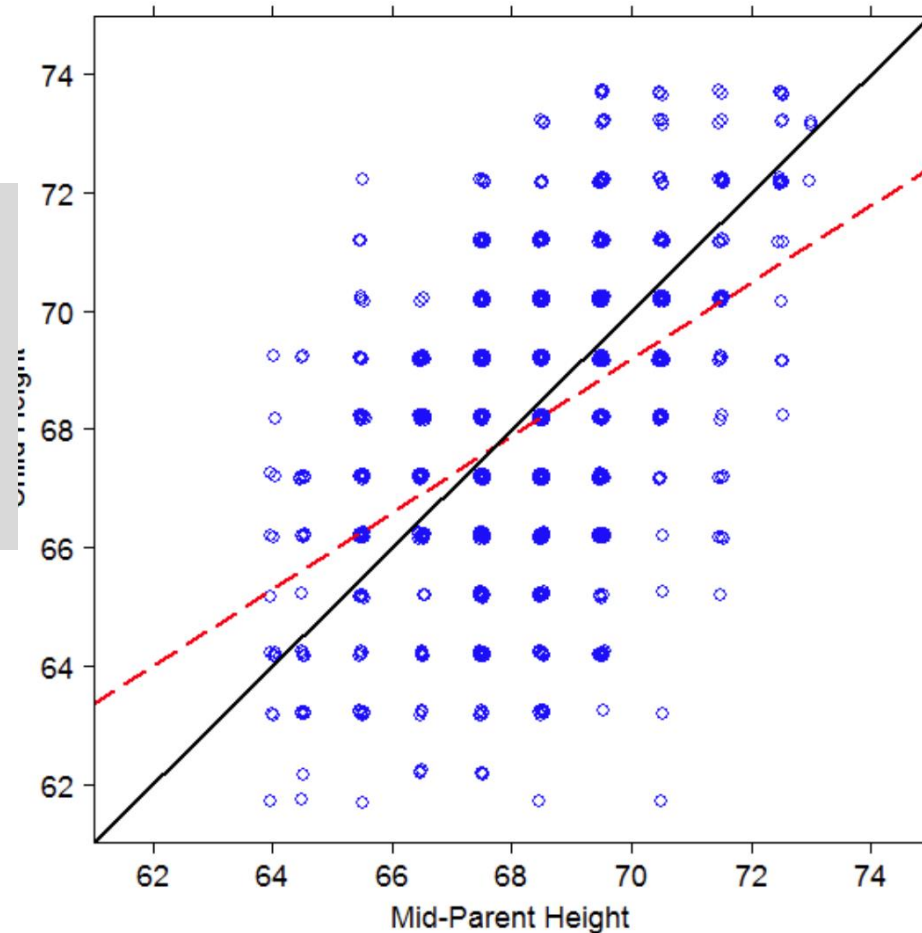
Sir Francis Galton (1822-1911) *"regression to the mean"*.



"It appeared from these experiments that the

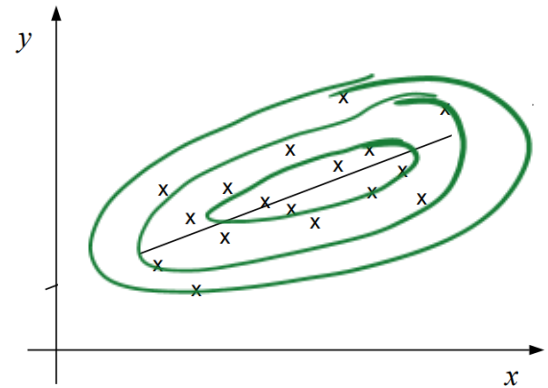
Don't confuse this meaning of the term "regression" with that in ML/statistics.

smaller than the parents, if the parents were large; to be larger than the parents, if the parents were small."



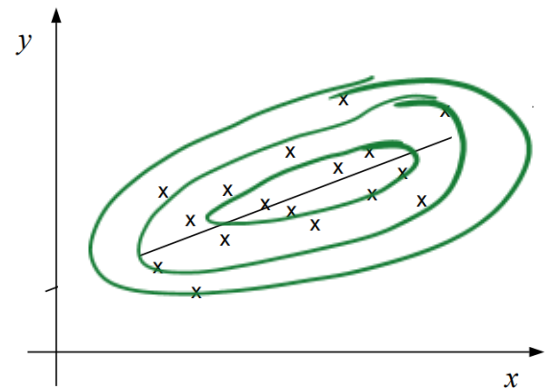
Possible Regression Tactics (goal: estimate $p(y|x)$)

- Our data are drawn from some distribution, $(X, Y) \sim p(x, y)$.
 - What are possible strategies to estimate $p(y|x)$?
1. Estimate $p(x, y|\theta)$ e.g. MVG for RVs X, Y , and then use fitted model to compute $p(y|x, \hat{\theta})$



Possible Regression Tactics (goal: estimate $p(y|x)$)

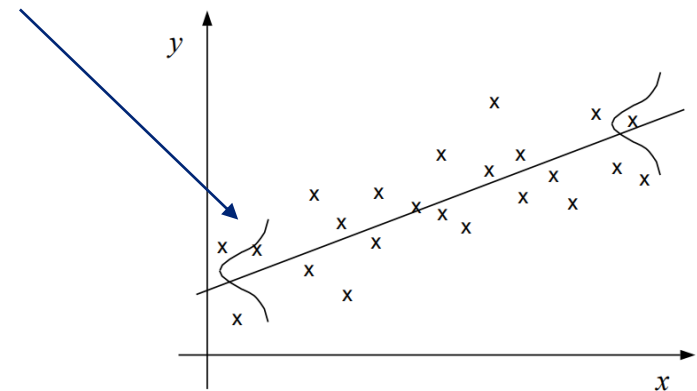
- Our data are drawn from some distribution, $(X, Y) \sim p(x, y)$.
 - What are possible strategies to estimate $p(y|x)$?
1. Estimate $p(x, y|\theta)$ e.g. MVG for RVs X, Y , and then use fitted model to compute $p(y|x, \hat{\theta}) = \frac{p(y, x|\hat{\theta})}{p(x|\hat{\theta})} = \frac{p(y, x|\hat{\theta})}{\int_y p(y, x|\hat{\theta}) dy}$.



Possible Regression Tactics (goal: estimate $p(y|x)$)

- Our data are drawn from some distribution, $(X_i, Y_i) \sim p(x, y)$.
 - What are possible strategies to estimate $p(y|x)$?
1. Estimate $p(x, y|\theta)$ e.g. MVG for RVs X, Y , and then use fitted model to compute $p(y|x, \hat{\theta}) = \frac{p(y, x|\hat{\theta})}{p(x|\hat{\theta})} = \frac{p(y, x|\hat{\theta})}{\int_y p(y, x|\hat{\theta}) dy}$.
 2. Consider the inputs to be fixed, and model only the output as a RV. That is, directly model the conditional $p(y|x, \hat{\theta})$.

"generative", vs. "discriminative"



Linear Regression

- Takes the discriminative approach.
- Predictions are a linear function of the parameters:

$$\hat{y} = E_Y[p(y|x)] = \mathbf{w}^T \mathbf{x} + \mathbf{w}_0, \text{ for } \mathbf{w}, \mathbf{x} \in \mathbb{R}^d.$$

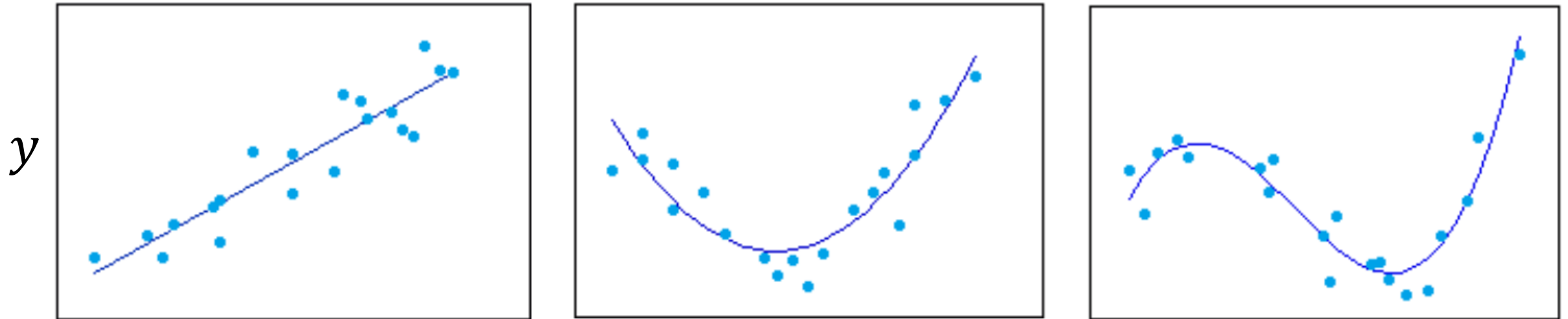
- $\mathbf{w}_0$ is called the "offset"/"bias"/"intercept".

Book-keeping trick: instead of a bias, we can make an extra feature that is always **1**, now use $\mathbf{x}' = [\mathbf{x}, 1]$ and $\hat{y} = \mathbf{w}^T \mathbf{x}'$.

How useful can a linear model be?!

Which of these curves could have been modelled by linear regression?

$$\hat{y} = E_Y[p(y|x)] = \mathbf{w}^T \mathbf{x} + w_0, \text{ for } \mathbf{w}, \mathbf{x} \in \mathbb{R}^d$$

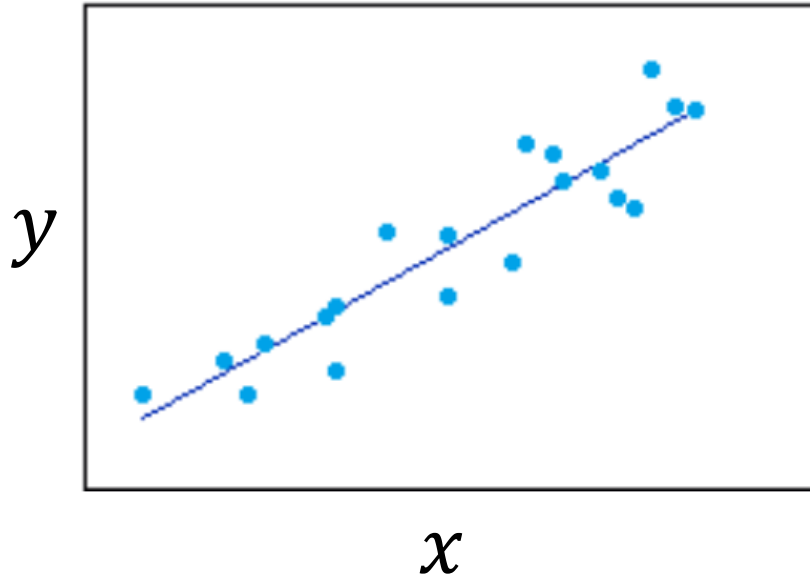


How useful can a linear model be?!

$$w, x \in \mathbb{R}^1$$

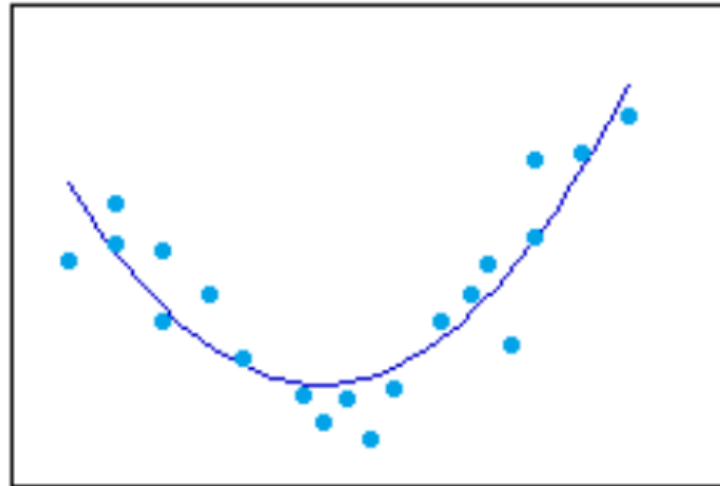
$$\hat{y} = w^T x$$

Linear



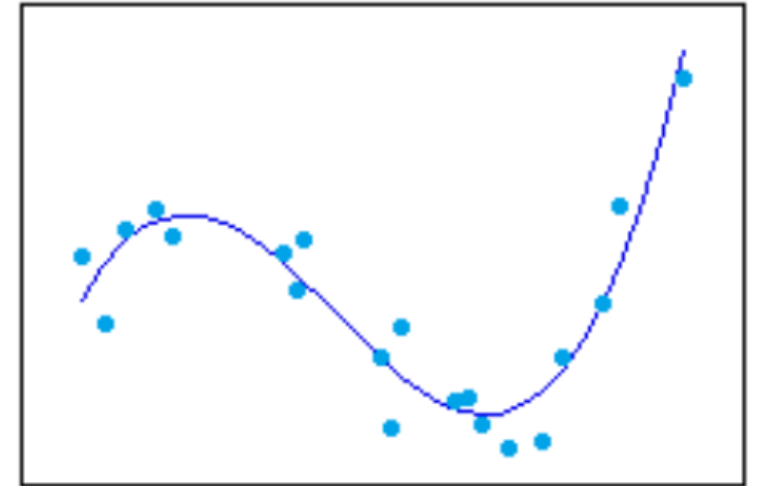
$$\hat{y} = w^T [x, x^2]$$

Quadratic



$$\hat{y} = w^T [x, x^2, x^3]$$

Cubic



For full generality, $x \in \mathbb{R}^D$ need the cross-terms and bias terms for arbitrary polynomial, e.g., quadratic $[1, x_1, x_2, x_1^2, x_2^2, x_1 x_2]$.

Basis expansion of raw input space

$$x \in \mathbb{R}^{d=2} = [x_1, x_2] \rightarrow [1, x_1, x_2, x_1 x_2, x_1^2, x_2^2] \in \mathbb{R}^{k=6}$$

Polynomial expansion of order 2 (i.e. quadratic)

- Denote *basis expansion* of the (raw) input features:
 $\Phi(x): \mathbb{R}^d \rightarrow \mathbb{R}^k$.

For $d = 1$, some polynomial expansions are:

- A quadratic expansion ($k = 2$), $\Phi(x) = [1, x, x^2]$.
- A cubic expansion ($k = 3$), $\Phi(x) = [1, x, x^2, x^3]$.

Identity basis expansion, $\Phi(x) = x$, and $k = d$.

Basis expansion of raw input space

Basis functions are pre-determined, so just a notational change:

$$\hat{y} = E_Y[p(y|x)] = \mathbf{w}^T \Phi(\mathbf{x}), \text{ for } \mathbf{w} \in \mathbb{R}^k, \mathbf{x} \in \mathbb{R}^d$$

In this lecture, for simplicity of notation, we will assume that this expansion has already been done, and just write $\hat{y} = \mathbf{w}^T \mathbf{x}$.

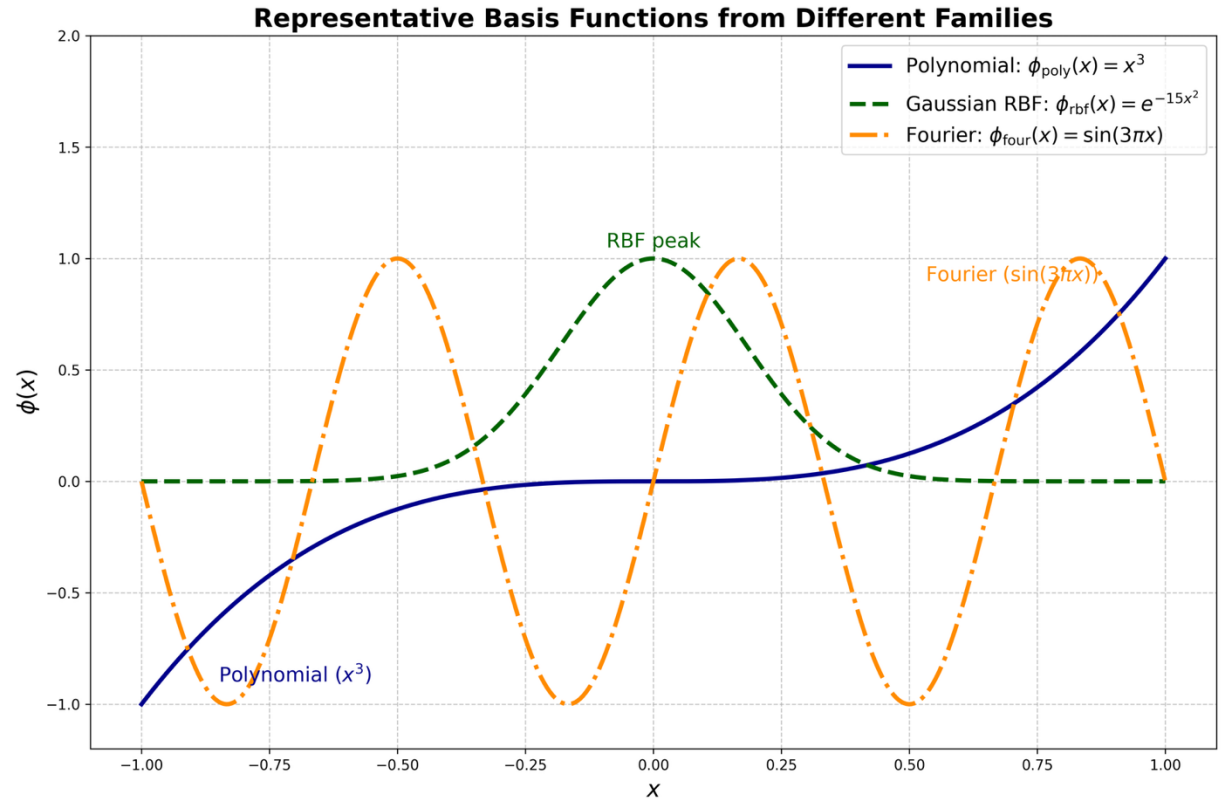
Many basis possible functions!

Examples in 1D:

- Polynomial basis: $\phi_j(x) = x^j$
- Radial Basis Functions (RBF):

$$\phi_j(x) = e^{-\frac{(x-\mu_j)^2}{r}}$$

- Sinusoidal basis: $\phi_j(x) = \sin x_j$
or $\phi_j(x) = \cos x_j$
- Fourier...
- Wavelets...
- *etc.*

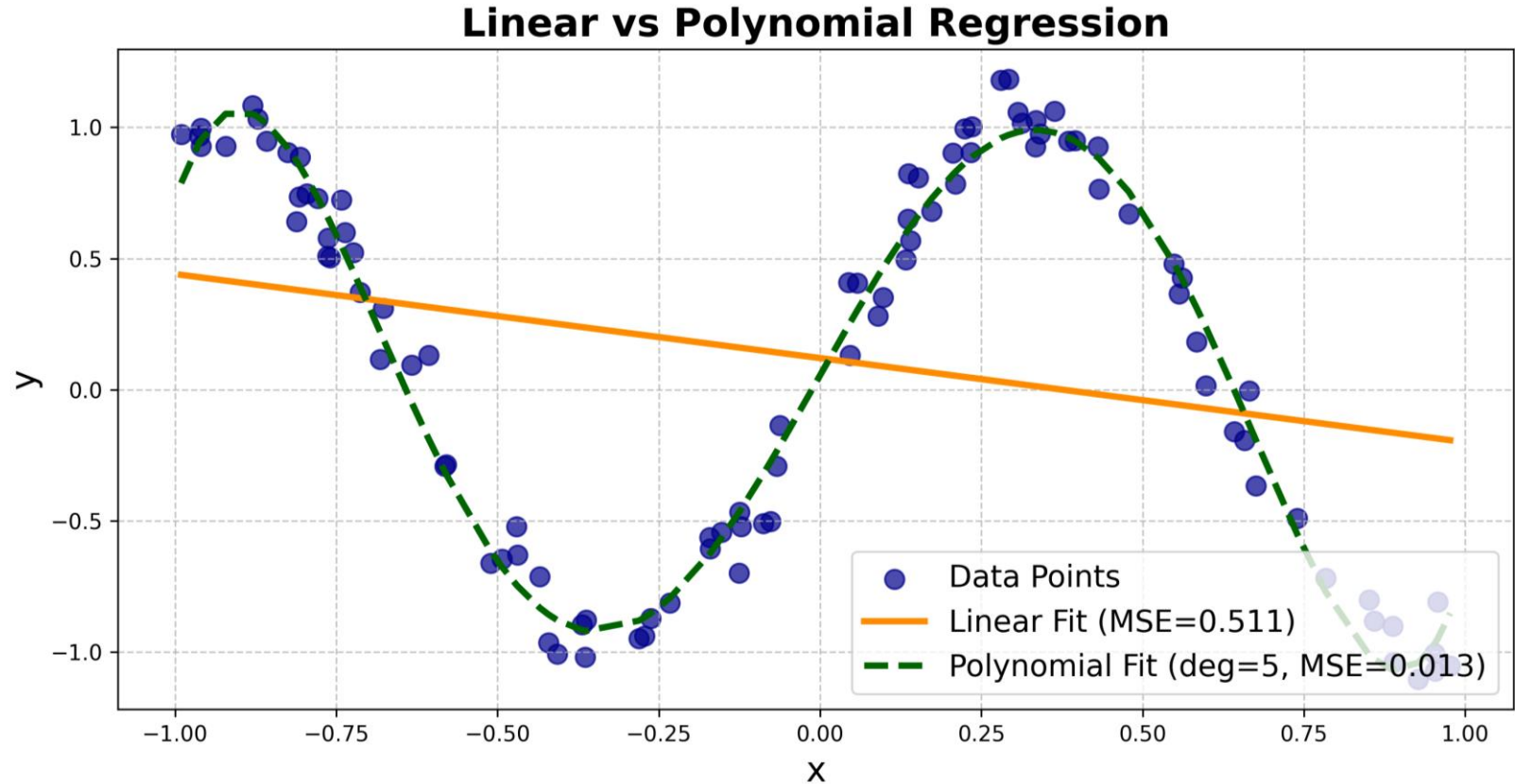


Basis expansion can add complexity to modeling

Example in 1D:

Using polynomial basis functions of degree 5, we redefined the linear regression equation as:

$$y = f(\vec{x}, |\vec{w}|) = \sum_{j=0}^5 w_j x^j$$

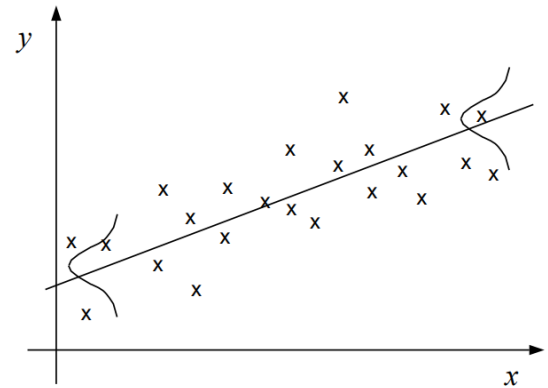


$$y(x) = 0.000 + 4.355x^1 - 0.203x^2 - 14.426x^3 + 0.164x^4 + 9.368x^5$$

$$y(x) = 0.120 - 0.320x$$

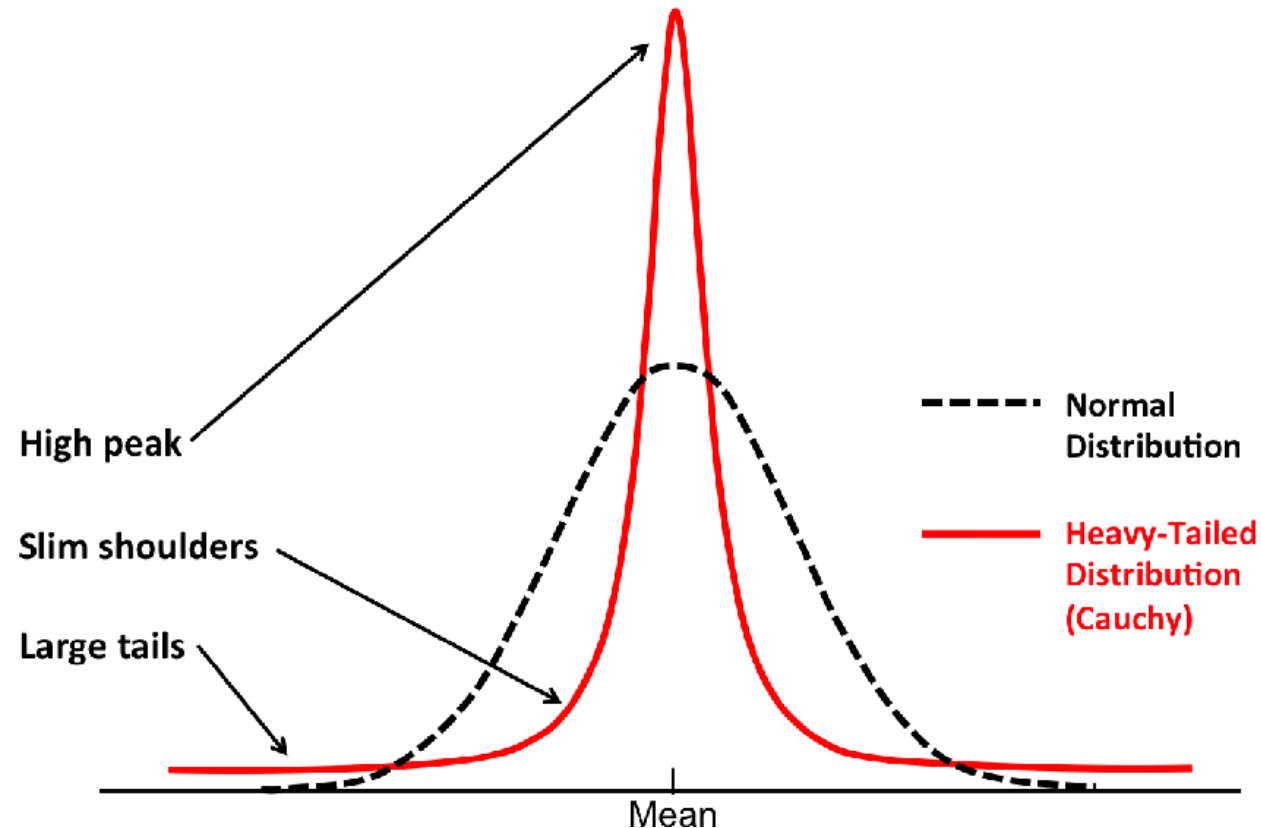
Specific form of linear regression

- So far we said linear regression is $\hat{y} = E_Y[p(y|x)] = E[p(y|x)] = \mathbf{w}^T \mathbf{x}$.
- But what do we use for $p(y|x)$?
- Standard linear regression uses a Gaussian $p(y|x) = N(y|\mathbf{w}^T \mathbf{x}, \sigma^2)$.
- Equivalent to $Y = \mathbf{w}^T \mathbf{x} + \epsilon$, with $\epsilon \sim N(0, \sigma^2)$.
- Which is equivalent to $Y - \mathbf{w}^T \mathbf{x} = \epsilon \sim N(0, \sigma^2)$.
- Alternate forms give “heavier tails” to the distribution of the “residual”.



Aside: heavy-tailed distribution

- More of the mass lies further from the center of mass.
- Heavy-tailed noise better models outliers than a Gaussian.

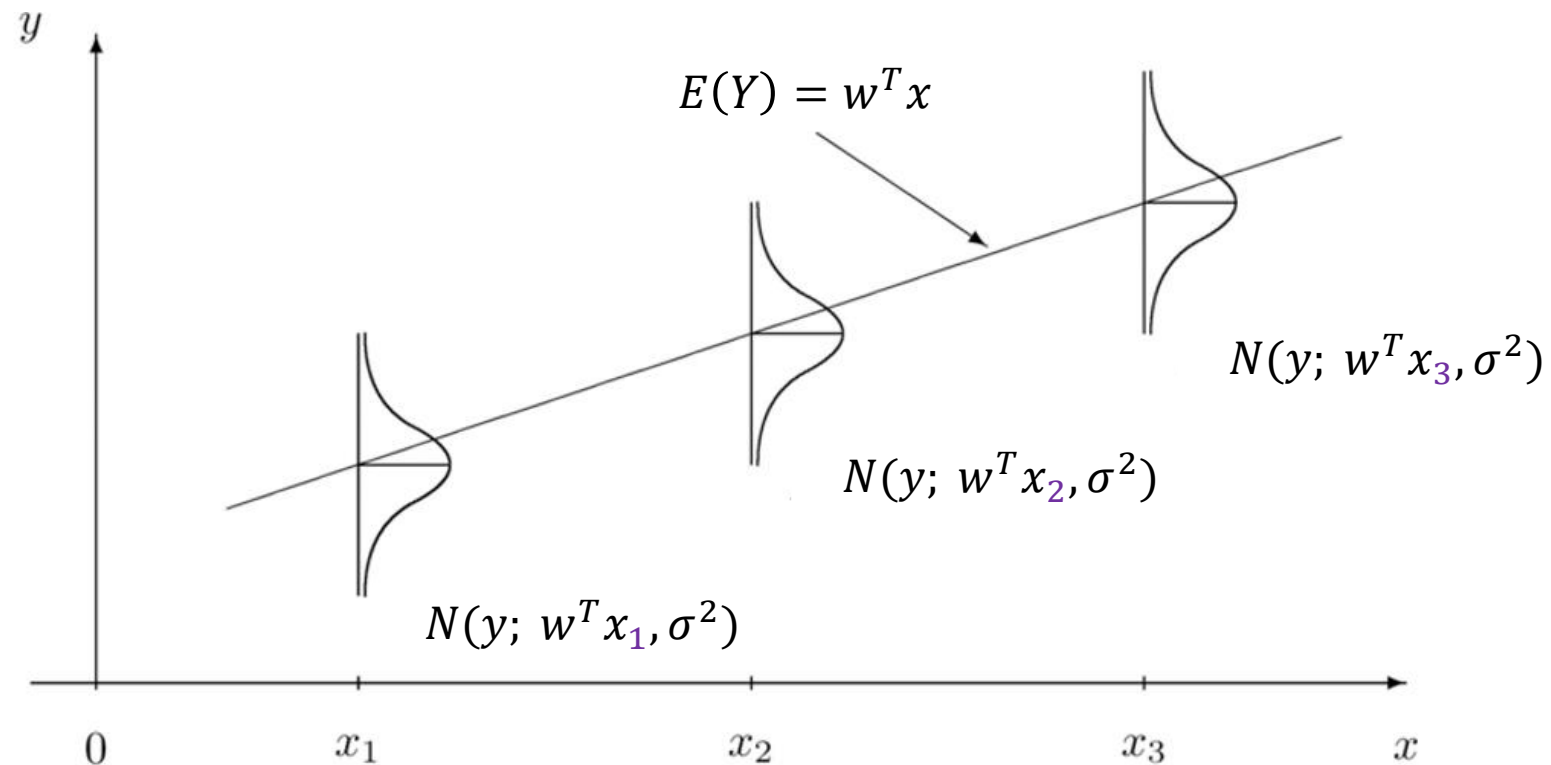


$$Y - w^T x = \epsilon \sim N(0, \sigma^2)$$

$$Y - w^T x = \epsilon \sim \text{Cauchy}(0, \sigma^2)$$

Gaussian linear regression, $p(y|x) = N(y|w^T x, \sigma^2)$

For every value $X = x$, the target variable, Y , takes on a Gaussian distribution with the same variance, σ^2 :



“Training” a Gaussian linear regression model

How will we fit the regression model, $p(y|x) = N(y|\mathbf{w}^T x, \sigma^2)$?

$$\text{MLE: } \theta_{MLE} = (w_{MLE}, \sigma_{MLE}^2) = \arg \max_{(w, \sigma^2)} \log p(D = \{(x_i, y_i)_{i=1}^n\} | \theta)$$

$$= \arg \max_{(w, \sigma^2)} \sum_{i=1}^n \log p(y_i | x_i, \theta)$$

$$= \arg \max_{(w, \sigma^2)} \sum_{i=1}^n \log N(y_i | w^T x_i, \sigma^2)$$

$$= \arg \max_{(w, \sigma^2)} \sum_{i=1}^n \log \left(\frac{1}{\sqrt{2\pi\sigma^2}} \right) - \frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - w^T x_i)^2$$

$$= \arg \max_{(w, \sigma^2)} n \log \left(\frac{1}{\sqrt{2\pi\sigma^2}} \right) - \frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - w^T x_i)^2$$

"Training" a Gaussian linear regression model

$$p(y|x) = N(y|w^T x, \sigma^2)$$

Note, if we ignore σ_{MLE}^2

$$(w_{MLE}, \sigma_{MLE}^2) = \arg \max_{(w, \sigma^2)} n \log \left(\frac{1}{\sqrt{2\pi\sigma^2}} \right) - \frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - w^T x_i)^2$$

... then estimating w above is the same as

$$= \arg \min_w \sum_{i=1}^n (y_i - w^T x_i)^2$$

$$= \arg \min_w \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

"least squares" loss function!

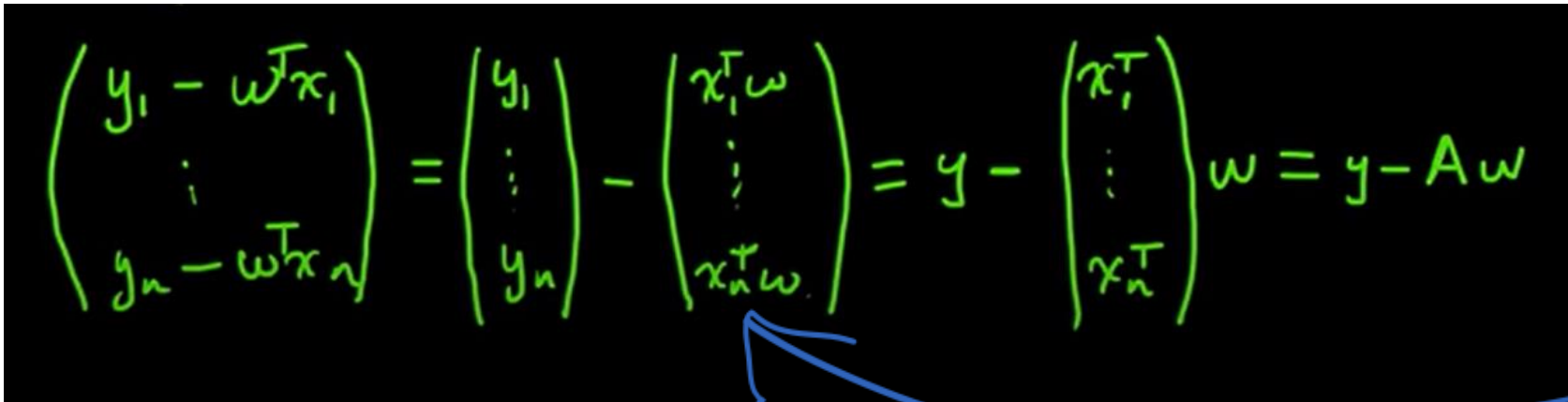
i.e., least squares arises naturally from conditional Gaussian MLE

"Training" a Gaussian linear regression model

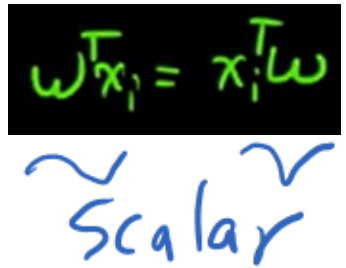
$$w_{\text{MLE}} = \arg \min_w \sum_{i=1}^n (y_i - w^T x_i)^2$$

Lets re-write this loss in "vectorized" form:

First, define:



A handwritten derivation on a black background showing the vectorization of the loss function. The equation is:
$$\begin{pmatrix} y_1 - w^T x_1 \\ \vdots \\ y_n - w^T x_n \end{pmatrix} = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix} - \begin{pmatrix} x_1^T w \\ \vdots \\ x_n^T w \end{pmatrix} = y - \begin{pmatrix} x_1^T \\ \vdots \\ x_n^T \end{pmatrix} w = y - A w$$
 A blue arrow points from the term $x_i^T w$ in the second vector to the boxed equation $w^T x_i = x_i^T w$ on the right.



A handwritten equation in a black box: $w^T x_i = x_i^T w$. Below the box, the word "Scalar" is written with a checkmark above it. A blue arrow points from this box to the term $x_i^T w$ in the main derivation.

"Training" a Gaussian linear regression model

$$w_{\text{MLE}} = \arg \min_w \sum_{i=1}^n (y_i - w^T x_i)^2$$

$$A \in \mathbb{R}^{n \times d}$$

Lets re-write this loss in "vectorized" form:

First, define:

$$\begin{pmatrix} y_1 - w^T x_1 \\ \vdots \\ y_n - w^T x_n \end{pmatrix} = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix} - \begin{pmatrix} x_1^T w \\ \vdots \\ x_n^T w \end{pmatrix} = y - \begin{pmatrix} x_1^T \\ \vdots \\ x_n^T \end{pmatrix} w = y - A w$$

$$w^T x_i = x_i^T w$$

Scalar

A is called
the "design
matrix"

"Training" a Gaussian linear regression model

$$w_{\text{MLE}} = \arg \min_w \sum_{i=1}^n (y_i - w^T x_i)^2$$

$$A \in \mathbb{R}^{n \times d}$$

Lets re-write this loss in "vectorized" form:

First, define:

$$\begin{pmatrix} y_1 - w^T x_1 \\ \vdots \\ y_n - w^T x_n \end{pmatrix} = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix} - \begin{pmatrix} x_1^T w \\ \vdots \\ x_n^T w \end{pmatrix} = y - \begin{pmatrix} x_1^T \\ \vdots \\ x_n^T \end{pmatrix} w = y - Aw$$

$$w^T x_i = x_i^T w$$

Scalar

Then, we can re-write the loss as

$$\begin{aligned} & \arg \min_w (y - Aw)^T (y - Aw) \\ &= \arg \min_w \|y - Aw\|_2^2 \end{aligned}$$

A is called
the "design
matrix"

“Training” a Gaussian linear regression model

So want to minimize this wrt w :

$$\begin{aligned}\mathcal{L} &= \|y - Aw\|_2^2 & (y \in \mathbb{R}^n, A \in \mathbb{R}^{n \times d}, w \in \mathbb{R}^{d \times 1}) \\ &= (y - Aw)^T (y - Aw) \\ &= (y^T - (Aw)^T)(y - Aw) \\ &= y^T y - w^T A^T y - y^T A w + w^T A^T A w \\ &= y^T y - 2w^T A^T y + w^T A^T A w\end{aligned}$$

$$\begin{pmatrix} x_1^T \\ \vdots \\ x_n^T \end{pmatrix}$$

A

To minimize, we want to set $\frac{\partial \mathcal{L}}{\partial w} = 0$.

Most easily achieved by using the rules of vector calculus, so lets do a quick refresher.

$$p(y|x) = N(y|w^T x, \sigma^2)$$

Refresher on vector calculus

Some “rules” for taking gradients with respect to vectors.

- e.g., for vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^{d \times 1}$, such that $\mathbf{a}^T \mathbf{b} \in \mathbb{R}$,

$$\frac{\partial(\mathbf{a}^T \mathbf{b})}{\partial a_j} = \frac{\partial(a_1 b_1 + a_2 b_2 + \cdots a_d b_d)}{\partial a_j} = b_j \in \mathbb{R}$$

Thus,

$$\frac{\partial(\mathbf{a}^T \mathbf{b})}{\partial \mathbf{a}} = \frac{\partial(a_1 b_1 + a_2 b_2 + \cdots a_d b_d)}{\partial \mathbf{a}} = \mathbf{b} \in \mathbb{R}^{d \times 1}$$

(not true for $\mathbf{a} \mathbf{b}^T$ which is a matrix, be careful!)

$$= \frac{\partial(\mathbf{b}^T \mathbf{a})}{\partial \mathbf{a}}$$

Refresher on vector calculus

For vector $x \in \mathbb{R}^{d \times 1}$, and matrix $\Sigma \in \mathbb{R}^{d \times d}$

$$\frac{\partial x^T \Sigma x}{\partial x} = (\Sigma + \Sigma^T)x$$

Thus, if Σ is symmetric such that $\Sigma = \Sigma^T$ then

$$\frac{\partial x^T \Sigma x}{\partial x} = 2\Sigma x$$

(similar to the scalar version: $\frac{\partial(ax^2)}{\partial x} = 2ax$)

"Training" a Gaussian linear regression model

So want to minimize

$$\mathcal{L} = (y - Aw)^T (y - Aw)$$

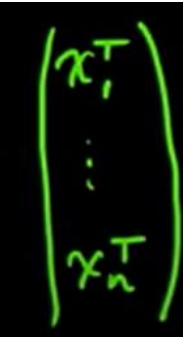
$$(y \in \mathbb{R}^n, A \in \mathbb{R}^{n \times d}, w \in \mathbb{R}^{d \times 1})$$

$$= (y^T - (Aw)^T)(y - Aw)$$

$$= y^T y - w^T A^T y - y^T A w + w^T A^T A w$$

$$= y^T y - 2w^T A^T y + w^T A^T A w$$

To minimize, we want to set $\frac{\partial \mathcal{L}}{\partial w^T} = 0$.



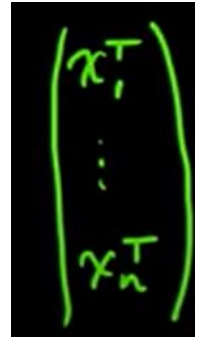
A handwritten green matrix notation for A , showing a column vector of x_1^T , a vertical ellipsis, and x_n^T . To the right, in purple, is the text $= A \in \mathbb{R}^{n \times d}$.

"Training" a Gaussian linear regression model

So want to minimize

$$\mathcal{L} = (y - Aw)^T (y - Aw)$$

$$(y \in \mathbb{R}^n, A \in \mathbb{R}^{n \times d}, w \in \mathbb{R}^{d \times 1})$$



Handwritten green vector notation: $\begin{pmatrix} x_1^T \\ \vdots \\ x_n^T \end{pmatrix}$. To its right, handwritten purple text: $= A \in \mathbb{R}^{n \times d}$.

$$= (y^T - (Aw)^T)(y - Aw)$$

$$= y^T y - w^T A^T y - y^T A w + w^T A^T A w$$

$$= y^T y - 2w^T A^T y + w^T A^T A w$$

To minimize, we want to set $\frac{\partial \mathcal{L}}{\partial w^T} = 0$.

$$\nabla_w \mathcal{L} = \begin{bmatrix} \partial \mathcal{L} / \partial w_1 \\ \partial \mathcal{L} / \partial w_2 \\ \vdots \\ \partial \mathcal{L} / \partial w_d \end{bmatrix} = \begin{bmatrix} -2A^T y + 2A^T A w \end{bmatrix}$$
$$= \underbrace{(-A^T y + A^T A w)}_{d \times 1} \cdot 2$$

"Training" a Gaussian linear regression model

So want to minimize

$$\begin{aligned}\mathcal{L} &= (y - Aw)^T (y - Aw) \\ &= (y^T - (Aw)^T)(y - Aw) \\ &= y^T y - w^T A^T y - y^T A w + w^T A^T A w \\ &= y^T y - 2w^T A^T y + w^T A^T A w\end{aligned}$$

To minimize, we want to set $\frac{\partial \mathcal{L}}{\partial w} = 0$.

$$\nabla_w \mathcal{L} = \begin{bmatrix} \frac{\partial \mathcal{L}}{\partial w_1} \\ \frac{\partial \mathcal{L}}{\partial w_2} \\ \vdots \\ \frac{\partial \mathcal{L}}{\partial w_n} \end{bmatrix} = \begin{bmatrix} -2A^T y + 2A^T A w \end{bmatrix}$$
$$= \underbrace{(-A^T y + A^T A w)}_{d \times 1} \cdot 2$$

"left pseudo-inverse" of A
 $(A^T A)^{-1} A^T * A = I$

$$A^+ A = I$$

$$\begin{pmatrix} x_1^T \\ \vdots \\ x_n^T \end{pmatrix} = A \in \mathbb{R}^{n \times d}$$

set to 0

rectangular
so no A^{-1} !

$$-A^T y + A^T A w = 0$$

$$A^T A w = A^T y$$

$$w = (A^T A)^{-1} A^T y$$

"Training" a Gaussian linear regression model

- A^+ is the Moore-Penrose inverse.
- Can be used even when A is not full rank.
- i.e., when A has dependent feature vectors.
- Will yield w_{MLE} with min. 2-norm, $\|w_{MLE}\|_2$

"left pseudo-inverse" of A
 $(A^T A)^{-1} A^T A = I$

$$A^+ A = I$$

$$\begin{pmatrix} x_1^T \\ \vdots \\ x_n^T \end{pmatrix}$$

$A \in \mathbb{R}^{n \times d}$

rectangular
so no A^{-1} !

$$-A^T y + A^T A w = 0$$

$$A^T A w = A^T y$$

$$w = (A^T A)^{-1} A^T y$$

"Training" a Gaussian linear regression model

- We still need to check if the critical point, $w = (A^T A)^{-1} A^T y$ is minimum of the squared error loss.
 - Recall $\nabla_w \mathcal{L} = -2A^T y + 2A^T A w$
 - So Hessian matrix ($\nabla_w^2 \mathcal{L}$) is $2A^T A$. When is $A^T A$ PD?
 - When the features in data set are independent (when it has full rank).
- ❖ N.B. σ^2 from MLE is just the mean squared residual, $\sigma^2 = \frac{1}{N} \sum_i (y_i - w^T x)^2$.

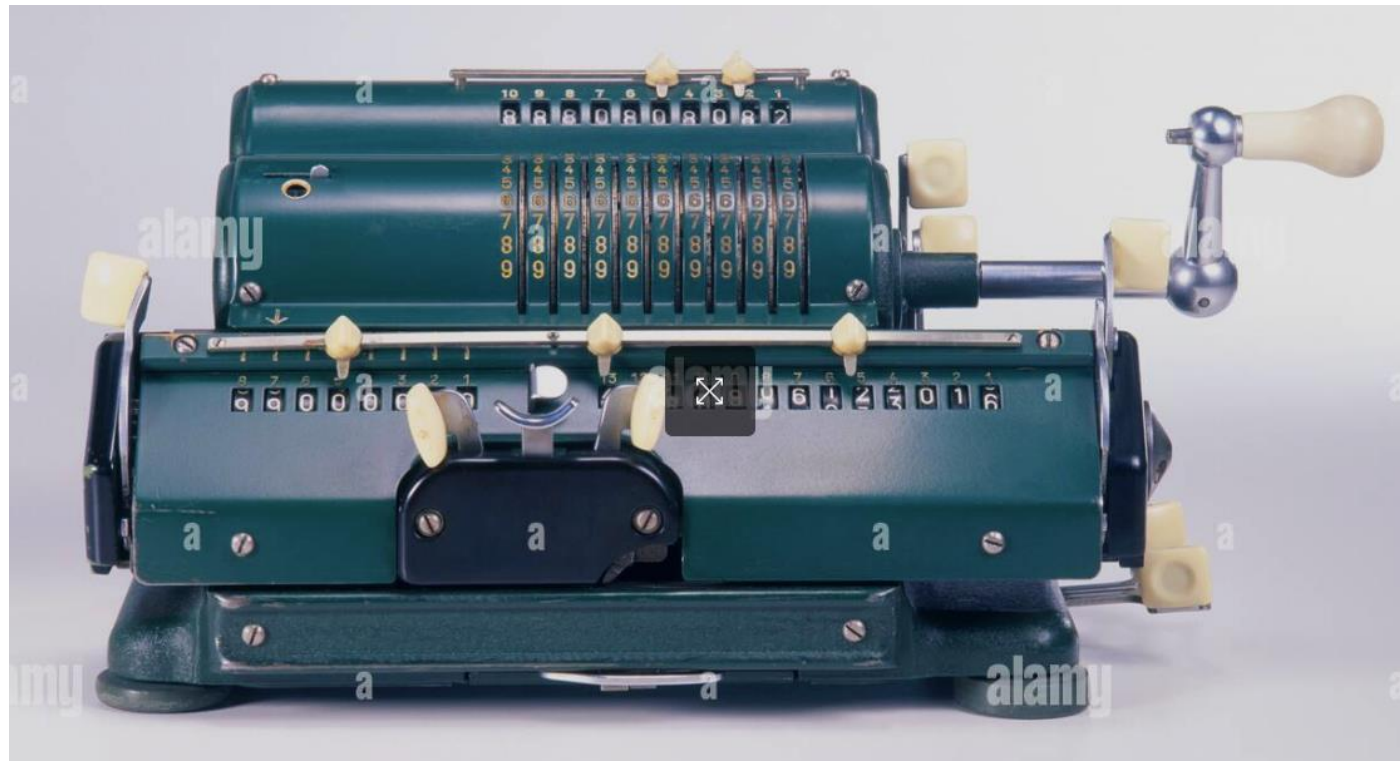
$$\begin{pmatrix} x_1^T \\ \vdots \\ x_n^T \end{pmatrix} = A \in \mathbb{R}^{n \times d}$$

$$\nabla_w \mathcal{L} = \begin{bmatrix} \partial \mathcal{L} / \partial w_1 \\ \partial \mathcal{L} / \partial w_2 \\ \vdots \\ \partial \mathcal{L} / \partial w_d \end{bmatrix} = \begin{bmatrix} -2A^T y + 2A^T A w \end{bmatrix}$$
$$= \underbrace{-A^T y + A^T A w}_{dx1}$$

$$\begin{aligned} -A^T y + A^T A w &= 0 \\ A^T A w &= A^T y \\ \boxed{w} &= (A^T A)^{-1} A^T y \end{aligned}$$

Regression in 1950s

Electromechanical desk "calculators" were used, and it could take up to 24 hours to receive the result from one regression on a small data set.



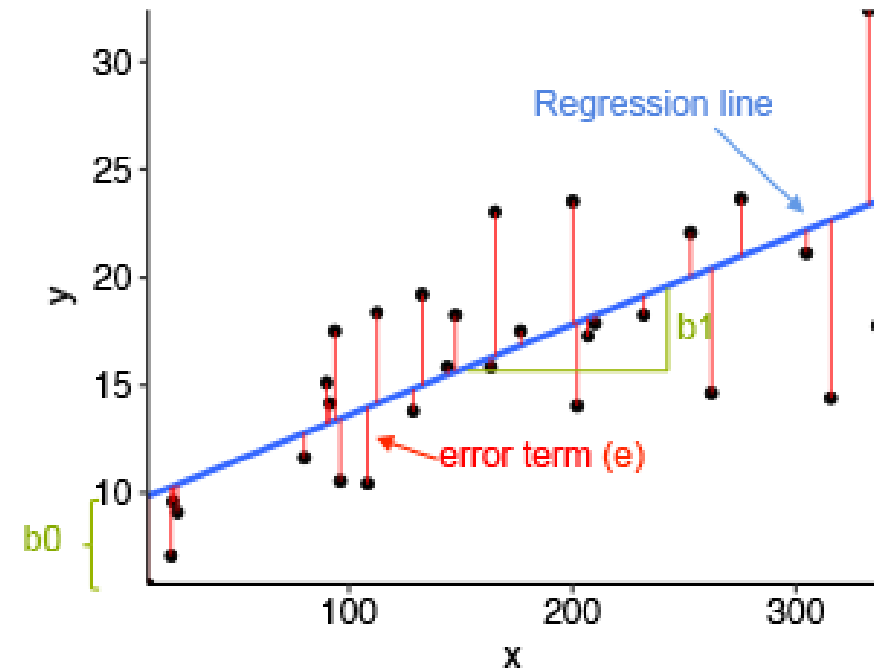
Geometric view of linear regression

- We're trying to predict all our training data labels correctly, such that $y_i = w^T x_i$ for all $i \in [1 \dots n]$.
- In vector form, this means we're looking for

$$\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} = A w = \begin{bmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_n^T \end{bmatrix} w$$

Generally, this is not possible because of noise; or incorrect model (e.g. missing some features).

$$\begin{pmatrix} x_1^T \\ \vdots \\ x_n^T \end{pmatrix} = A$$



Using basis expansions instead of x_i

$$\begin{pmatrix} x_1^T \\ \vdots \\ x_n^T \end{pmatrix} = A$$

- $\{x_j\} \rightarrow \{\Phi(x_j)\}$?
- Just define $\mathbf{A}$ with $\Phi^T(\mathbf{x}_j)$ instead of $\mathbf{x}_j$ because $\Phi(\mathbf{x})$ is fixed ahead of time, so its like someone just gave us different raw inputs $\mathbf{x}$.

$$\mathbf{x} = \Phi[x_1, x_2] = [1, x_1, x_2, x_1x_2, x_1^2, x_2^2] \in \mathbb{R}$$

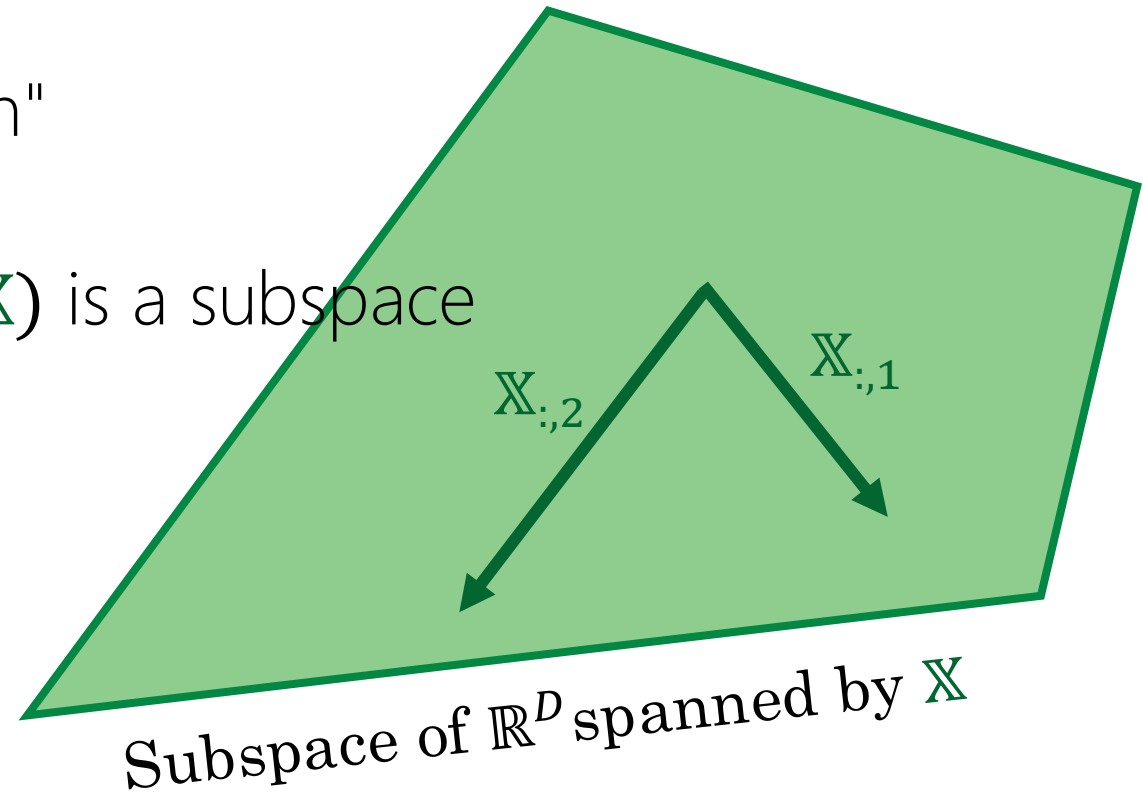
Polynomial expansion of order 2 (i.e. quadratic)

Roadmap

- Mixture of Gaussians
- Linear Regression
 - Probabilistic/MLE view
 - Geometric view

[Linear Algebra] Span

- The set of all possible linear combinations of the columns of $\mathbf{X}$ is called the **span** of $\mathbf{X}$ (denoted $\text{span}(\mathbf{X})$), also called the **column space**.
- Intuitively, this is all vectors you can "reach" using the columns of $\mathbf{X}$.
- If each column of $\mathbf{X}$ has length D , $\text{span}(\mathbf{X})$ is a subspace of $\mathbb{R}^D$.



[Linear Algebra] Matrix-Vector Multiplication

- **Approach 1:** So far, we've thought of our model as horizontally stacked predictions per datapoint:

$$\begin{bmatrix} | \\ \mathbb{Y} \\ | \end{bmatrix} = \begin{bmatrix} \text{---} & \mathbf{x}_1^T & \text{---} \\ \text{---} & \mathbf{x}_2^T & \text{---} \\ & \vdots & \\ & \mathbf{x}_N^T & \end{bmatrix} \begin{bmatrix} w \end{bmatrix}$$

$$Y = Aw$$

Matrix sizes

$$N \times 1 = (N \times d) (d \times 1)$$

- **Approach 2:** However, it is helpful sometimes to think of matrix-vector multiplication as performed by columns. We can also think of $\mathbb{Y}$ as a **linear combination of feature vectors**, scaled by **parameters**.

[Linear Algebra] Matrix-Vector Multiplication

- **Approach 1:** So far, we've thought of our model as horizontally stacked predictions per datapoint:

$$\begin{bmatrix} | \\ \mathbb{Y} \\ | \end{bmatrix} = \begin{bmatrix} \text{---} & \mathbf{x}_1^T & \text{---} \\ \mathbf{X} & \mathbf{x}_2^T & \text{---} \\ & \vdots & \\ \text{---} & \mathbf{x}_N^T & \text{---} \end{bmatrix} \begin{bmatrix} w \end{bmatrix}$$

Matrix sizes

$$N \times 1 = (N \times d) (d \times 1)$$

$$\mathbb{Y} = \mathbf{A} \mathbf{w}$$

- **Approach 2:** Helpful to think of matrix-vector multiplication as a **linear combination of feature vectors**, scaled by **parameters**.

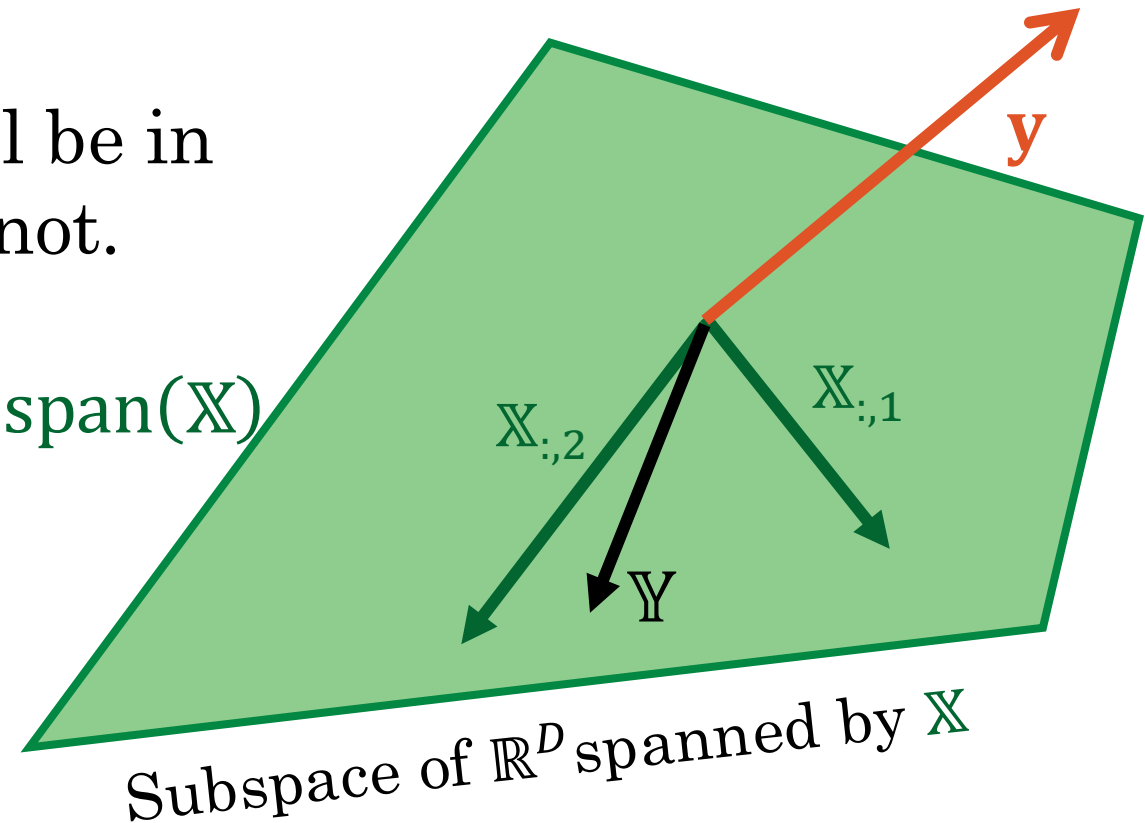
$$\begin{bmatrix} | \\ \mathbb{Y} \\ | \end{bmatrix} = \begin{bmatrix} | & & | \\ \mathbb{X}_{:,0} & \dots & \mathbb{X}_{:,d} \\ | & & | \end{bmatrix} \begin{bmatrix} w \end{bmatrix} = w_0 \mathbb{X}_{:,0} + w_1 \mathbb{X}_{:,1} + \dots + w_d \mathbb{X}_{:,d} = \sum_j w_j \mathbb{X}_{:,j}$$

Prediction Is a Linear Combination of Columns

Our prediction of $\mathbb{Y} = \mathbb{X} \vec{w} = \sum_j w_j \mathbb{X}_{:,j}$ is a linear combination of columns of $\mathbb{X}$.

Interpret: Our linear prediction $\mathbb{Y}$ will be in $\text{span}(\mathbb{X})$, even if observed values $\mathbf{y}$ are not.

Goal: Find vector of $\mathbb{Y}$ (predictions) in $\text{span}(\mathbb{X})$ that is closest to $\mathbf{y}$.

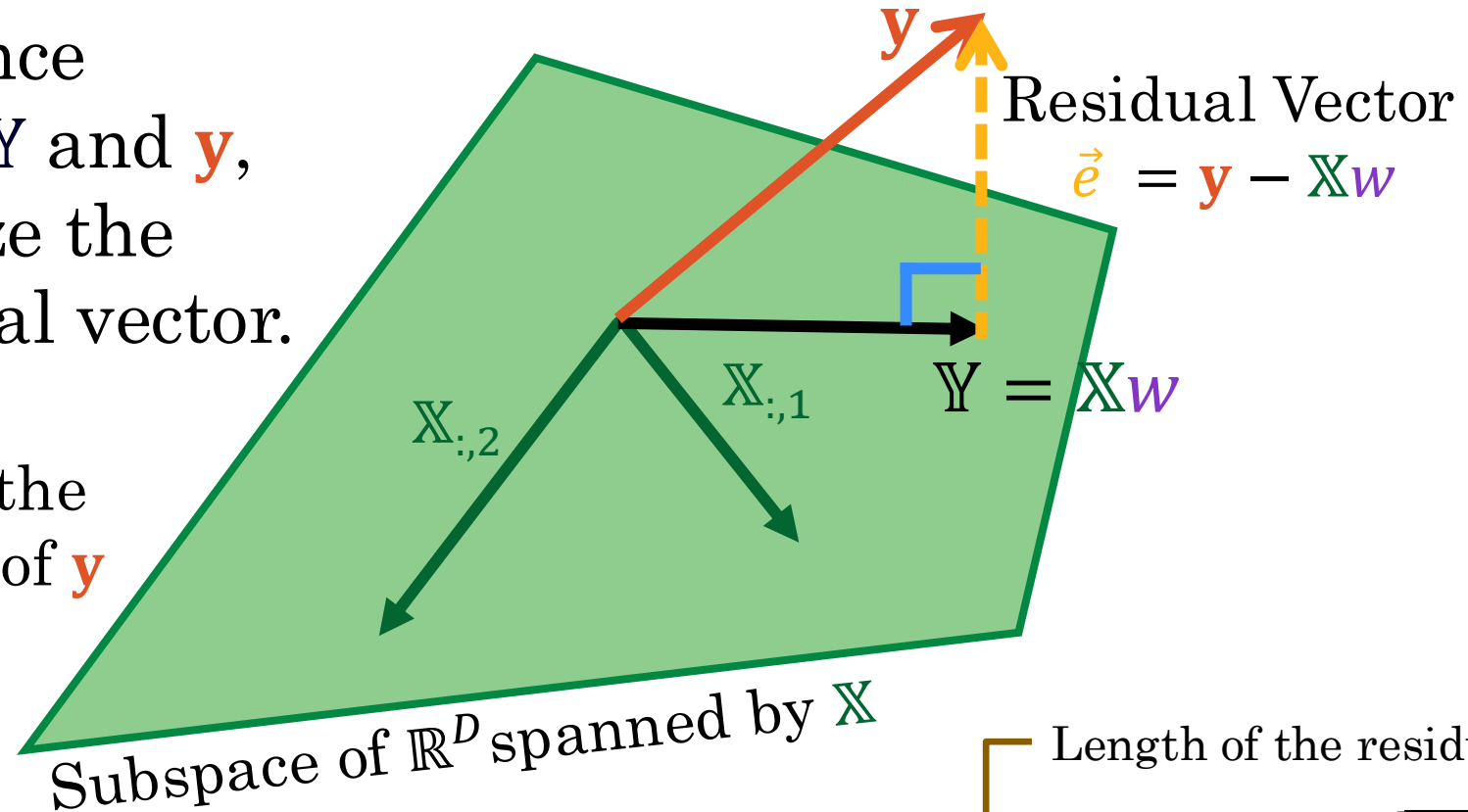


Finding Optimum Predictions ($\mathbb{Y}$)

$$\mathbb{Y} = \mathbb{X}w$$

To minimize distance between vector of $\mathbb{Y}$ and $\mathbf{y}$, we should minimize the norm of the residual vector.

$\vec{e}$ is minimized if it is the orthogonal projection of $\mathbf{y}$ on the $\text{span}(\mathbb{X})$.



Length of the residual vector

$$\|\vec{e}\|_2 = \sqrt{\sum_{d=0}^D e_d^2}$$

Finding Optimum Predictions ($\mathbb{Y}$)

Vector $\vec{e}$ is orthogonal to the $\text{span}(\mathbb{X})$, meaning that:

$$\mathbb{X}^T \vec{e} = 0$$

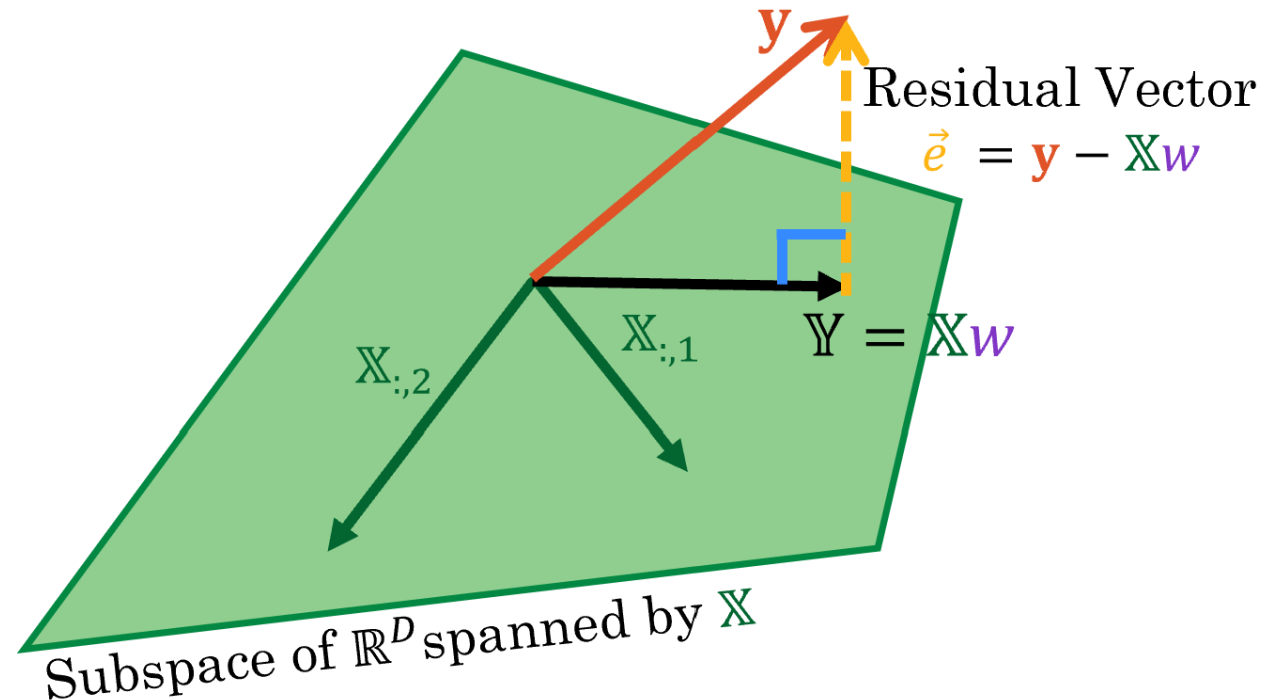
$$\mathbb{X}^T (\mathbf{y} - \mathbb{Y}) = 0$$

$$\mathbb{X}^T \mathbf{y} - \mathbb{X}^T \mathbb{X} \mathbf{w}^* = 0$$

$$\mathbf{y} = \mathbb{X}^T \mathbb{X} \mathbf{w}^*$$

➡ $\mathbf{w}^* = (\mathbb{X}^T \mathbb{X})^{-1} \mathbb{X}^T \mathbf{y}$

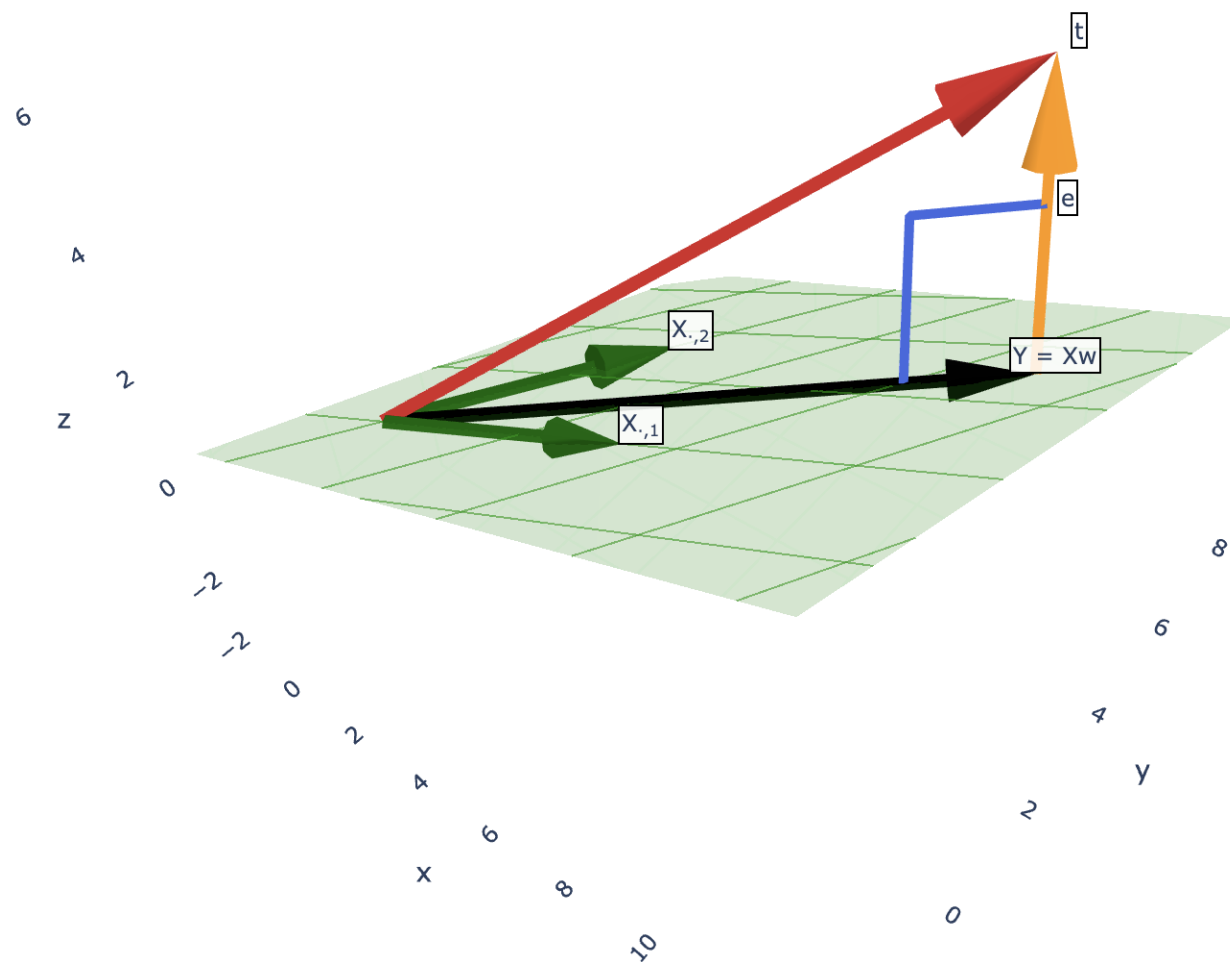
If $\mathbb{X}^T \mathbb{X}$ is invertible



$\vec{e}$ is minimized if it is the orthogonal projection of $\mathbf{y}$ on the $\text{span}(\mathbb{X})$.

Geometry of Least Squares in Plotly

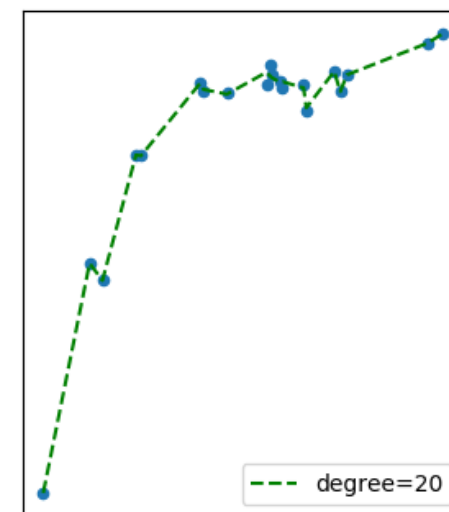
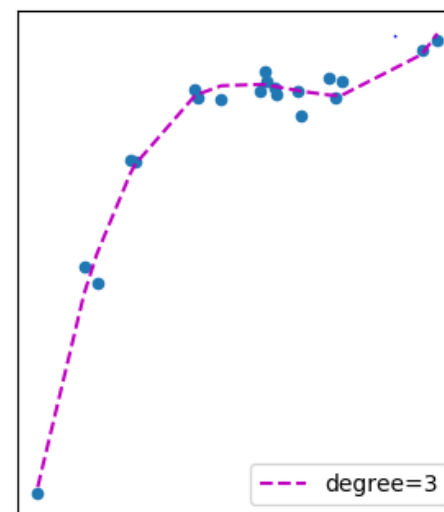
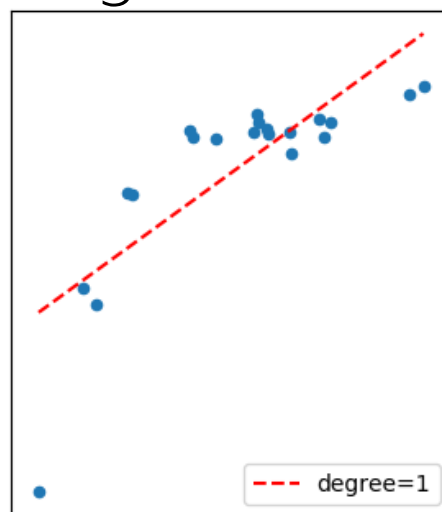
[Interactive link](#)



What can go wrong in linear regression

As we add higher and higher order polynomials, a few things happen:

1. # features, d gets bigger and bigger.
2. For $d \geq n$ can perfectly fit any data (i.e., polynomials are a complete basis).
3. Even when we don't perfectly fit the training data, we are still in danger of overfitting (worse prediction on test set). Our goal is not to fit a line through the training data exactly, it is to do well on unseen test cases!



What can go wrong in linear regression

Two main categories of fixes:

1. Remove features until the problem is well-behaved (“feature selection”).
2. Leave the features as they are, but *add constraints to the system* to “tighten it up” (aka “regularization”)—next class.

A thought experiment

Consider:

- Use MLE on data, $D = \{x_i, y_i\}$ to get $\hat{y}_i = p_{\theta}(y|\Phi(x))$ using linear regression.
- Assume an abundance of data (millions of data points), and only 100 parameters.
- Suppose get accuracy $\mp \$1000$ of sale price when applying to held out part of our data.
- Can we assume this model will get $\mp \$1000$ on any test set that may come in the future?

Actual vs. predicted sale price of house

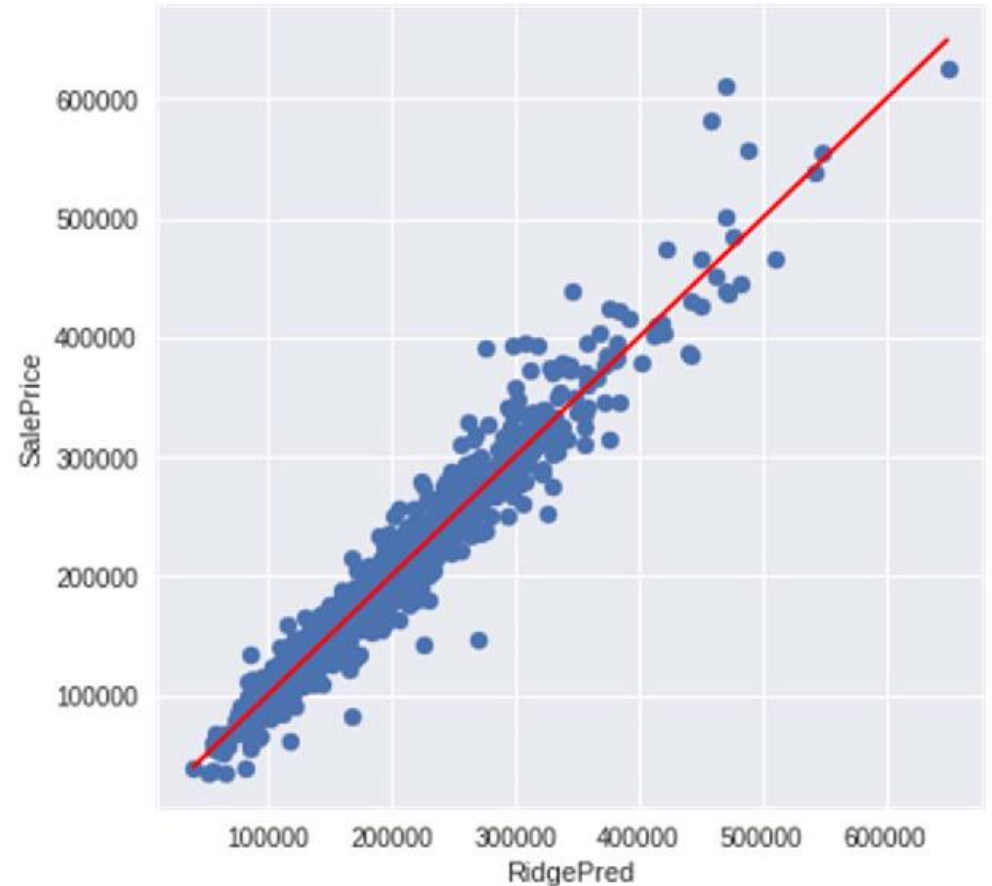


Fig. 4 Ridge Prediction for Training Data.

Causation vs correlation

Breakingviews

Zillow's failed house flipping

Reuters

WSJ NOV. 2021 : *“The company expects to record losses of more than \$500 million from home-flipping by the end of this year and is laying off a quarter of its staff.”*

Actual vs. predicted sale price of house

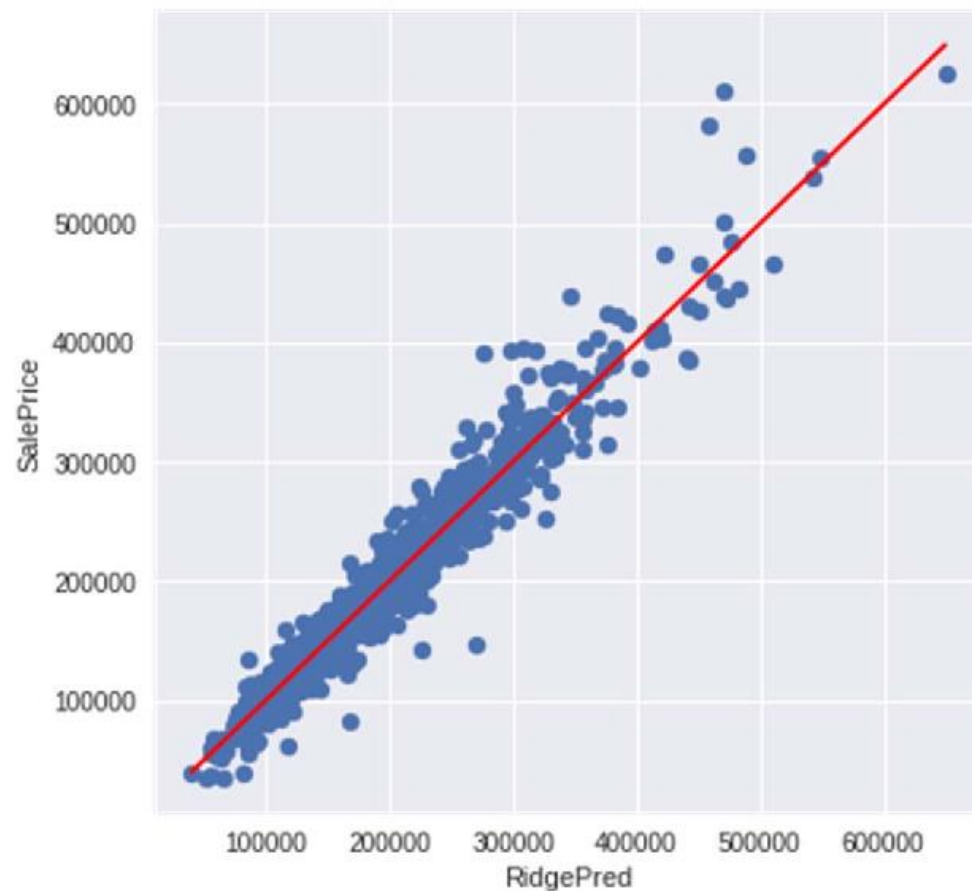


Fig. 4 Ridge Prediction for Training Data.